

CTF 网络安全入门：Crypto 专攻版

先识别表示，再找可逆关系

零基础版 • 题目 C0-C4（含玄机 586 五格电文、587 联考回执）

本册特色

- 零基础预备篇：字节、十六进制、异或、模运算、RSA 三元组，每个术语先讲人话；
- 每题一张标准题面卡：给什么、要什么、交付物、考点、前置知识、难度；
- 逐题图解带做：每一步命令配真实终端截图，看到图就能照着做；
- 所有截图来自真实执行（termcap 实录 + 终端渲染），所有 flag 来自真实运行输出；
- 附命令速查表、术语表、自测题与完整推演。

适用对象：学过 Python 和命令行、第一次接触 CTF 的同学

版本：2026 年 9 月

使用说明

这本讲义带你做什么

Crypto（密码学）方向的题目，说到底只有两种任务：把看不懂的数据还原成看得懂的数据，或者把一段关系想明白。本册用 5 道题把最常见的三类突破口各练一遍：数据表示（C0）、异或与穷举（C1）、RSA 的数学关系（C2-C4）。做完之后，你应当能自己回答三个问题：手里这份数据到底是什么表示？它的密钥空间有多大？题目给了什么可验证的关系？

学习路线图

1. **第 1 章零基础预备**：装好 Python 3 和 xxd，跑通第一条命令，确认你能看到和讲义一样的输出；
2. **第 2 章 Crypto 基础知识铺垫**：字节与十六进制、异或真值表、模运算、RSA 三元组 (n,e,c) 、小模数与共模的危险、自定义编码的方法论。这章不写题，但每一小节都对应后面某道题的某一步；
3. **第 3 章 Crypto 工具箱**：python 一行命令、xxd、pow 三参数快速幂与求逆元，每个工具配真实输出截图；
4. **第 4 章题目：题面卡与逐题图解带做**：每题一张题面卡，卡后面直接就是这道题的解题步骤，按“读题与思路 → 分步操作与观察 → 结果与核对 → 为什么会成功 → 排错指南 → 变式练习”推进。建议先遮住解题步骤自己做一遍，卡住再看；
5. **第 6、7 章**：复盘知识地图与易错清单，做自测题并对照完整推演；
6. **附录**：玄机平台真题状态与提交 checklist、命令速查表、术语表。

怎么用这本讲义

- **先看图再敲命令**：每张终端截图都标了这一步在做什么。截图里的路径以本资料根目录为当前目录，例如 `labs/crypto/xor.hex`。

- **每步都问“为什么”**：正文里每条命令后面都紧跟一句“这一步是为了……”。能自己说出为什么，才算会做。
- **核对不能省**：解出 flag 后一定要做“结果与核对”里的独立验证。脚本打印出 `flag{...}` 不等于答案正确。
- **变式练习是升级关**：做对原题只是及格，做完变式才算掌握。

安全声明

所有实验只针对本资料附带的程序和明确允许练习的平台；不要把命令用于未经授权的系统。玄机平台入口为 <https://xj.edisec.net/>，授课时由教师提供个人或课程账号，讲义不保存账号口令。平台题面、价格与界面可能变化，始终以当前题目详情页为准。

截图说明

本册所有终端截图都来自真实执行：先用 `tools/termcap.py` 在真实 shell 里执行命令并记录“命令 + 真实输出”，再用 `tools/term2png.py` 渲染成终端窗口图。每张图对应的原始实录保存在 `figures/raw/` 下的同名 `.txt` 文件里，可自行核对。原理示意图（异或真值表、RSA 流程图、五格电文比特流图）由 matplotlib 绘制，画的是结构关系。玄机平台的网页没有伪造截图，涉及平台状态的地方用文字表格说明。

术语约定

每个术语第一次出现时都会用白话解释，之后直接使用。最容易混淆的三个词先放在这里：

- **字符**：人眼看到的一个字，如字母 A、汉字“夜”；
- **字节**：计算机存储的一个 0-255 的数，是数据的最小常用单位；
- **十六进制**：字节的一种显示方式，一个字节显示成两个字符，如字节 65 显示成 41。

目录

使用说明	i
第一章 零基础预备：实验环境与第一次动手	1
1.1 Crypto 题长什么样	1
1.2 安装与验证（Windows 侧）	1
1.3 安装与验证（WSL 侧，可选）	2
1.4 目录与文件约定	3
1.5 第一次运行脚本	3
1.6 用独立证据验证答案	4
第二章 Crypto 基础知识铺垫	5
2.1 计算机里的数据：字符、字节、十六进制	5
2.2 异或 XOR：真值表与“加密 = 解密”	8
2.3 为什么单字节异或可以穷举	8
2.4 模运算：余数的世界	9
2.5 RSA 从头讲：三元组 (n, e, c) 与私钥 d	10
2.6 小模数为什么危险：试除分解	12
2.7 共模攻击为什么成立：同一 n 、两个 e	12
2.8 自定义符号编码：五格电文的方法论	14
2.9 本章知识点速查	15
第三章 Crypto 工具箱	16
3.1 python3：交互、一行命令与脚本	16
3.2 xxd：看文件的十六进制视图	17
3.3 pow 三参数：快速幂与求逆元	18

3.4	读附件的三件小工具	19
3.5	工具箱小结	20
第四章	题目：题面卡与逐题图解带做	21
4.1	题 C0：十六进制字符串到底有多少字节?	21
4.2	题 C1：单字节异或如何从密文找回明文?	23
4.3	题 C2：五格电文的符号到底表示什么?（玄机 586）	27
4.4	题 C3：小模数 RSA 为什么能被试除?	31
4.5	题 C4：同一模数的两份回执能否互相补全?（玄机 587）	35
第五章	复盘：知识地图与易错清单	40
5.1	解题路线图：拿到材料先做什么	40
5.2	下次遇到这类题怎么办	41
5.3	易错清单（按“错一下浪费半小时”的顺序）	41
第六章	自测与参考答案	43
6.1	课后自测题	43
6.2	参考答案与完整推演	43
6.3	完整推演：C3 的每一步为何必要	45
6.4	完整推演：C4 的推导为何成立	45
6.5	完整推演：C2 的验证链为何必要	46
附录 A：	玄机平台真题状态与提交 checklist	47
6.6	本册涉及的平台题状态	47
6.7	提交 checklist	47
附录 B：	命令速查表	49
附录 C：	术语表	51

第一章 零基础预备：实验环境与第一次动手

本章目标：装好 Python 3 与 xxd；能在正确目录下运行命令；理解“文件内容”和“文件显示”不是一回事。

1.1 Crypto 题长什么样

Crypto 题的题面通常很短：给一段“看不懂的数据”，要一个 `flag{...}` 字符串。看不懂的数据可能是一串十六进制字符、一堆整数、一串奇怪符号，或者几个大整数。破解它们不靠猜，靠三步：

1. **识别表示：**这份数据现在是文本、字节、整数列表还是符号串？
2. **找可逆关系：**加密过程有没有留下结构？密钥空间小不小？有没有重复使用参数？
3. **验证：**还原结果能不能被独立证据确认（已知样本、再加密比对、平台反馈）？

本册的 5 道题分别训练这三步里的不同环节。请记住第一句话：**先识别表示，再找可逆关系。**

1.2 安装与验证（Windows 侧）

要装什么

- **Python 3** (3.8 以上即可)：所有解题脚本都用它。安装时勾选“Add python.exe to PATH”。
- **PowerShell**：Windows 自带，不用装。
- **WSL/Ubuntu** (可选，做 xxd 练习时用)：Microsoft Store 安装 Ubuntu-24.04。没有 WSL 也能做完本册全部题目，xxd 只是看十六进制的工具之一。

验证命令 在 PowerShell 里执行（当前目录切到本资料根目录）：

```
python --version
python -c "print('hello, Crypto!'); print(2+3)"
Get-ChildItem labs/crypto | Select-Object Name, Length
```

应该看到 Python 3.x 的版本号、两行打印，以及 labs/crypto 目录下的三个文件：make_challenges.py、rsa.txt、xor.hex。图 1.1 是一次真实执行的截图。

```
Windows env check: python + labs/crypto

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python --version
Python 3.12.10
PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'print('hello, Crypto!');
print(2+3)'
hello, Crypto!
5
PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> Get-ChildItem labs/crypto |
Select-Object Name, Length
Name                               Length
----                               -
make_challenges.py                 492
rsa.txt                            158
xor.hex                            46
```

图 1.1: 第 1 章 Windows 侧环境验证：python 版本、一行命令试跑、实验目录文件列表（真实终端实录）

一句话记忆

看到和讲义一样的输出，才说明环境对了；输出不一样时先别做题，先修环境。

1.3 安装与验证（WSL 侧，可选）

WSL 里的 Ubuntu 提供 xxd（十六进制查看工具）和 python3。验证命令（在 PowerShell 里执行）：

```
wsl -d Ubuntu-24.04 -- bash -lc "python3 --version"
wsl -d Ubuntu-24.04 -- bash -lc "xxd -v"
```

图 1.2 是真实执行结果：Ubuntu 里是 Python 3.12.3，xxd 为 2023-10-25 版。注意 Windows 的 C:\ 盘在 WSL 中通常位于 /mnt/c/，比如本资料根目录在 WSL 里是 /mnt/c/Users/你的用户名/Desktop/CTF 零基础自学资料包。

```
WSL env check: python3 + xxd

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && python3 --version
Python 3.12.3
mzj@Ubuntu-24.04:~$ xxd -v 2>&1 | head -1
xxd 2023-10-25 by Juergen Weigert et al.
mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && ls -l labs/crypto
总计 0
-rwxrwxrwx 1 mzj mzj 492  9月 24 10:09 make_challenges.py
-rwxrwxrwx 1 mzj mzj 158  9月 26 10:52 rsa.txt
-rwxrwxrwx 1 mzj mzj  46  9月 26 10:52 xor.hex
```

图 1.2: 第 1 章 WSL 侧环境验证: python3 与 xxd 版本、实验目录列表 (真实终端实录)

常见误区

Windows 的 python 和 WSL 的 python3 是两套环境。本册命令若没有特别说明, 都在 Windows PowerShell 里执行; 标注“WSL”的才去 WSL。

1.4 目录与文件约定

书中的相对路径以本资料根目录为当前目录。终端的当前目录不对时, 程序会报“找不到文件”, 这不代表题目坏了。先用 `pwd` (PowerShell 里是 `Get-Location`) 和 `ls` (PowerShell 里是 `Get-ChildItem`) 确认位置。本册用到的文件:

- `labs/crypto/xor.hex`: C1 的密文 (十六进制文本文件);
- `labs/crypto/rsa.txt`: C3 的小模数 RSA 参数与密文列表;
- `labs/crypto/make_challenges.py`: 生成上面两个文件的脚本, 可用于复现题目数据 (默认不要改动 `labs/` 下任何文件);
- `labs/platform/attachments/586-five-grid.zip`、`labs/platform/attachments/587-linked-receipts.zip`: 两道平台题的官方附件本地副本;
- `labs/platform/solve_five_grid.py`、`labs/platform/solve_receipt.py`: 两道平台题的复现脚本。

1.5 第一次运行脚本

在根目录运行平台题脚本 (现在只是“试跑”, 第 4 章会讲每一步的含义):

```
python labs/platform/solve_five_grid.py
python labs/platform/solve_receipt.py
```

如果两条命令都打印出 `flag: flag{...}` 字样, 说明 Python 环境、路径、附件副本全部就绪。看到 `flag` 不用记, 后面会带你一步步把它们做出来。

一句话记忆

脚本能跑通 = 环境就绪; 脚本报“找不到文件” = 当前目录不对; 脚本报编码错误 = 先加 `python -X utf8` 再试。

1.6 用独立证据验证答案

脚本打印一串 `flag{...}` 只说明脚本输出了字符串, 不等于答案正确。本册每个“结果与核对”模块都给出**独立于最后那行打印**的验证方法:

- C1: 把明文重新异或一遍, 看是否还原出原密文;
- C3: 用公钥把明文再加密一遍, 看是否等于密文列表;
- C2 (586): 先用公开试报解出已知明文“夜班 OK”, 映射对了才解正文;
- C4 (587): 用两个指数分别再加密, 两份都要对上;
- 平台题最后还要看提交页面是否显示“FLAG 正确”。

写题解时也应写出从附件条件到结果的证据链, 而非只附一行看似成功的输出。

第二章 Crypto 基础知识铺垫

本章目标：把字节、十六进制、异或、模运算、RSA 三元组、小模数与共模、自定义编码这七块地基打牢，每块都能对应到后面的具体题目。

2.1 计算机里的数据：字符、字节、十六进制

字符是人看到的字；字节是计算机存的一个 0-255 的数；十六进制是字节的一种显示方式。三者的关系是：字符 $\xrightarrow{\text{编码}}$ 字节 $\xrightarrow{\text{显示}}$ 十六进制文本。

ASCII 最古老的编码表：把 0-127 的数分配给英文字母、数字、标点。例如字母 A 对应 65（十六进制 41），小写 f 对应 102（66）。英文字符在 ASCII 里占 1 个字节。**UTF-8** 是目前最常用的编码：英文部分兼容 ASCII（1 字节），汉字通常占 3 字节。

动手看一眼 运行图 2.1 里的两条命令：第一条把字符 A 编码成字节再显示成十六进制；第二条把 flag{ 这五个字符逐个打印成十六进制。你应该看到：b'A' 41 1 表示字符 A 是 1 个字节、十六进制写作 41；b'41' 却是 2 个字节——那是字符 4 和字符 1！而 bytes.fromhex('41') 才是 1 个字节。

```
bytes, ASCII and hex are different views

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'text='A';
raw=text.encode('utf-8'); print(raw, raw.hex(), len(raw)); print(bytes.fromhex('41'));
print(b'41', len(b'41')); print(bytes.fromhex('4142'), len(bytes.fromhex('4142')));
print('flag'.encode('utf-8').hex())'
b'A' 41 1
b'A'
b'41' 2
b'AB' 2
666c6167

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'for ch in "flag{":
print(ch, hex(ord(ch)))'
f 0x66
l 0x6c
a 0x61
g 0x67
{ 0x7b
```

图 2.1: 第 2 章字节、ASCII 与十六进制的关系：字符 ↔ 字节 ↔ 十六进制的换算演示（真实终端实录）

最常踩的坑 把“十六进制文本”当成“字节本身”。文件里存着 44 个字符 515b...544a，它其实只是一段文本；用 `bytes.fromhex()` 解析后才得到 22 个字节。每跨过一种表示，都应写下转换方法和转换后的长度。

常见误区

`b'41'` 是 2 个字节（两个 ASCII 字符），`bytes.fromhex('41')` 是 1 个字节（值为 0x41）。数数字符个数前先想清楚手里的是哪种。

表示方式的候选假设 拿到一串看不懂的数据，先做假设再验证：

- 只包含 0-9a-fA-F 且长度为偶数：可能是字节的十六进制表示；
- 包含大小写字母、数字、斜线和加号，末尾有等号：可能是 Base64；
- 固定长度的长十六进制串（比如 32 位）：可能是哈希摘要；
- 由 32 个互不相同的怪符号组成：可能是自定义的 5 位编码（见 2.8 节）。

它们只是候选假设，不是确证。先试解析，再检查结果里有没有文件头、可打印文本或下一层编码。Base64 的转换规则公开、没有秘密密钥，所以它是**编码**，不是**加密**。

十六进制和 Base64 到底差在哪 两者都是“把字节换成一串字符”的编码，规则公开、没有密钥，都能无损还原；区别只在用哪些字符和膨胀多少：

- **十六进制**：1 个字节写成 2 个十六进制字符（长度变为 2 倍），字符表只有 0-9a-f；
- **Base64**：每 3 个字节写成 4 个可打印字符（长度约 1.33 倍），会出现大写、小写、数字和 + /，末尾常用 = 补齐。

识别时先看**字符表**，再看**长度比**。图 2.2 用同一份 9 字节数据演示：`.hex()` 得到 18 个字符，`base64.b64encode()` 得到 12 个字符，二者都能还原原始字节。以后在 C0、C1 里看到 0-9a-f 就想到十六进制，看到 + / = 就想到 Base64。

```
Base64 vs Hexadecimal display

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -X utf8
figures/raw/fig-crypto-25-b64-vs-hex.py

原始字节      : b'flag{hex}'  字节数 9
十六进制显示   : 666c61677b6865787d  字符数 18 （比值 2 倍）
Base64 显示    : ZmxhZ3toZXh9  字符数 12 （比值约 1.33 倍）
识别规律: hex 只出现 0-9a-f
              Base64 会出现大写、小写、数字和 + /，末尾常带 =
两者都能无损还原原始字节：
bytes.fromhex(hex)      -> b'flag{hex}'  一致：True
base64.b64decode(base64) -> b'flag{hex}'  一致：True
```

图 2.2: 第 2 章 Base64 与十六进制对照：同一份字节的两种显示、字符表与长度比的差别（真实终端实录）

常见误区

同一份字节，十六进制和 Base64 的字面完全不同（例如 666c6167 与 ZmxhZw== 是同一段内容），别把两者混用：用错解码函数会报错或解出乱码。判断“是不是 Base64”不能只看长度，关键看字符表里有没有 + / =。

一句话记忆

字符是给人看的，字节是给计算机存的，十六进制和 Base64 都是给眼睛看字节的方式：前者 2 倍长、只有 0-9a-f，后者约 1.33 倍长、含 + / =。跨表示先记长度。

2.2 异或 XOR：真值表与“加密 = 解密”

异或（XOR，符号 $\oplus$ ）是一个按位运算：两个位相同得 0，不同得 1。它的真值表和可逆性见图 2.3。

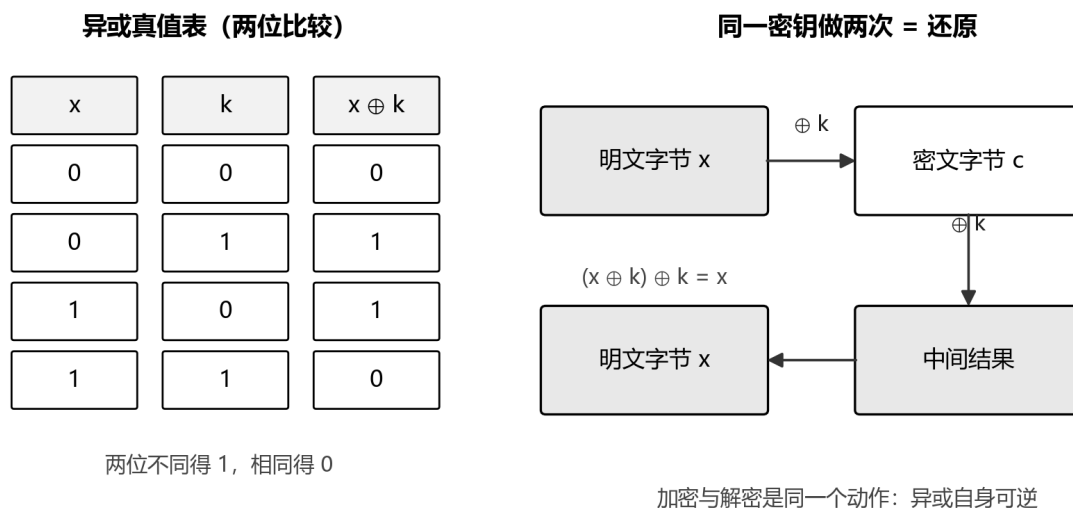


图 2.3: 第 2 章异或真值表与可逆性：两位比较规则，以及同一密钥做两次即还原（原理示意图）

为什么说“加密等于解密”？因为异或有个关键性质：

$$(x \oplus k) \oplus k = x$$

用密钥 k 异或一次是加密，再用同一个 k 异或一次就还原。所以异或加密的解密程序就是加密程序本身。Python 里逐字节异或写作 `bytes(x ^ k for x in data)`（ $\wedge$ 是 Python 的异或运算符）。

异或的局限 若密钥与消息等长、完全随机、只使用一次（一次性密码本），理论上不可破。但 CTF 题里密钥通常很短、反复使用：C1 就是 1 个字节的密钥循环作用于所有明文字节。此时密文的结构就泄漏了密钥的结构。

一句话记忆

异或一次是加密，异或两次是还原；密钥重复使用，结构就会泄漏。

2.3 为什么单字节异或可以穷举

密钥只有 1 个字节，取值范围 0–255，一共只有 **256 种可能**。哪怕不懂数学，把 256 种全试一遍也就试完了，这叫**穷举**（brute force）。难点只在于：试出来的 256 个结果里，哪一个是

真明文？判断方法叫**可读性判断**：

- **看前缀**：flag 通常以 `flag{` 开头；
- **看字符范围**：真明文多半是可打印字符（ASCII 32–126），乱码会混进控制字符；
- **看语言**：英文明文里字母、空格占比高；中文明文按 UTF-8 解码要合法；
- **看结构**：花括号配对、单词可读、长度合理。

把候选密钥、可读比例和解出的原始字节都打印出来，方便复核。第 4 章 C1 会完整走一遍。

常见误区

短数据可能有很多“看起来可读”的结果。只有一个字符被猜中不算数，至少要检查完整的 `flag{` 前缀和整段的字符范围。

2.4 模运算：余数的世界

模运算（mod）就是“除以 n 取余数”： $17 \bmod 5 = 2$ 。它有几个好用的性质：

- 结果永远落在 0 到 $n - 1$ 之间，再大的数取模后都变小；
- 加、乘、乘方都可以先算再取模，结果不变。这就是**快速幂**：算 $2^{100} \bmod 1000$ 不用先算出 31 位的 2^{100} ，Python 的 `pow(2, 100, 1000)` 三参数版本专门干这个；
- 在模 n 意义下，有的数有**逆元**：满足 $a \cdot a^{-1} \equiv 1 \pmod{n}$ 的那个 a^{-1} 。Python 直接写 `pow(a, -1, n)`。

用钟表理解“绕回来” 模运算可以想成看钟面：一圈 12 小时，15 点其实就是下午 3 点，写成 $15 \bmod 12 = 3$ ；再走满一圈又回到 3 点——这正是“结果只落在 0 到 $n - 1$ ，一超过就从 0 重新数”。负数也一样： $(-8) \bmod 5 = 2$ ，因为从 0 往回退 8 步，绕了 2 圈之后落回 2。有了这个直觉就能明白：RSA 里的指数无论多大，取模后都会被“绕”回一个小区间，所以快速幂能一直保持小数字、算得又快又准。图 2.4 是真实演示：注意 $-8 \bmod 5 = 2$ （Python 的余数永远非负），以及 2 的幂次模 5 会循环出现 1, 2, 4, 3。求逆元并不总是可行： a 与 n 不互素时逆元不存在。

最大公约数与“互素” 刚刚反复出现的“互素”需要先定义。**最大公约数** $\gcd(a, b)$ 指能同时整除 a 和 b 的最大正整数，Python 写作 `math.gcd(a, b)`，例如 `math.gcd(12, 18) = 6`。当 $\gcd(a, b) = 1$ （两数除 1 以外没有公共因子）时，称 a 与 b **互素**。互素是逆元存在的唯一前提：只有 $\gcd(a, n) = 1$ ，`pow(a, -1, n)` 才求得出逆元，否则会报 `ValueError`。“ e 与 $\varphi(n)$ 互素”这道 RSA 门槛，用的就是这个概念。

```
modular arithmetic: remainder and fast pow

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'print("17 % 5 =", 17 % 5); print("-8 % 5 =", (-8) % 5); print("pow(2,100) 的十进制位数:", len(str(pow(2,100)))); print("pow(2,100,1000) =", pow(2,100,1000)); print("pow(7,123,10**9+7) =", pow(7,123,10**9+7))'
17 % 5 = 2
-8 % 5 = 2
pow(2,100) 的十进制位数: 31
pow(2,100,1000) = 376
pow(7,123,10**9+7) = 937329259
PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'print("重复取模不会跑偏: 2 的幂次模 5 循环:", [pow(2,i,5) for i in range(10)])'
重复取模不会跑偏: 2 的幂次模 5 循环: [1, 2, 4, 3, 1, 2, 4, 3, 1, 2]
```

图 2.4: 第 2 章模运算演示: 余数、快速幂、循环性 (真实终端实录)

一句话记忆

模 n = 除以 n 取余数; 快速幂用 $\text{pow}(b, e, n)$; 求逆元用 $\text{pow}(a, -1, n)$, 前提是 a 与 n 互素。

2.5 RSA 从头讲: 三元组 (n, e, c) 与私钥 d

RSA 是最常见的公钥加密算法。CTF 题里你通常会拿到三个数: 模数 n 、加密指数 e 、密文 c , 合称三元组 (n, e, c) 。加密与解密流程见 图 2.5。

RSA: 三元组 (n, e, c) 与藏起来的私钥 d

拿到 (n, e, c) 不等于能解密; 要 d 就得知道 $\varphi(n)$ 或分解 n

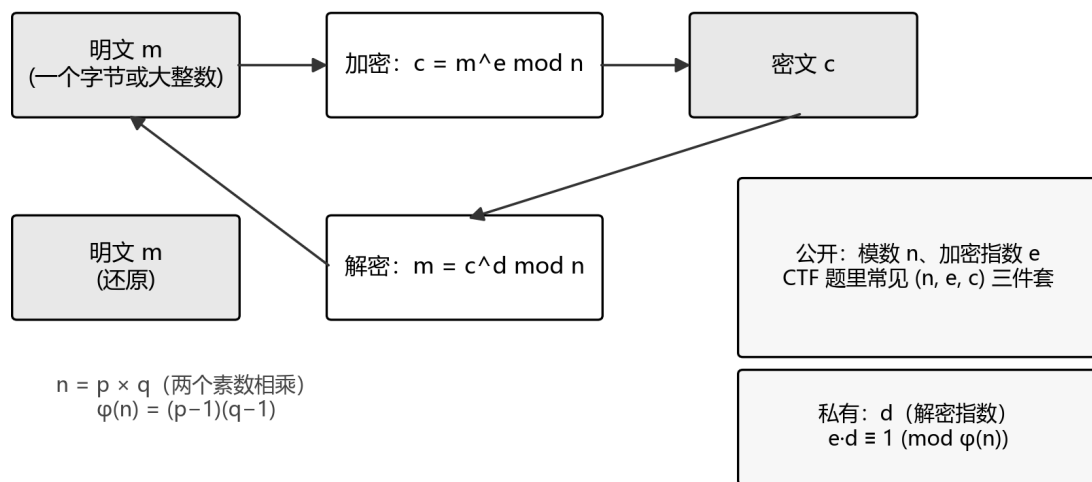


图 2.5: 第 2 章 RSA 加解密流程: 公钥 (n, e) 加密、私钥 d 解密, 以及 d 与 $\varphi(n)$ 的关系 (原理示意图)

一张图背后的四件事

1. $n = p \times q$: 模数是两个大素数的乘积。 p, q 是秘密;
2. 加密: $c \equiv m^e \pmod{n}$ 。任何人都能用公开的 (n, e) 加密;
3. 解密: $m \equiv c^d \pmod{n}$ 。需要私钥指数 d ;
4. d 怎么来: $\varphi(n) = (p-1)(q-1)$, d 是 e 模 $\varphi(n)$ 的逆元, 即 $e \cdot d \equiv 1 \pmod{\varphi(n)}$ 。

为什么 d 能解密 d 是 e 模 $\varphi(n)$ 的逆元, 意思是 $e \cdot d \equiv 1 \pmod{\varphi(n)}$, 也就是 $e \cdot d = 1 + k \varphi(n)$ 。解密就是把密文 $c = m^e$ 再取 d 次幂, 而指数相乘可以拆开:

$$c^d = (m^e)^d = m^{e \cdot d} = m^{1+k \varphi(n)} = m \cdot (m^{\varphi(n)})^k \equiv m \pmod{n}.$$

这里用到欧拉定理 $m^{\varphi(n)} \equiv 1 \pmod{n}$ (当 $\gcd(m, n) = 1$), 多出来的部分被“绕”成 $1^k = 1$, 最后只剩 $m^1 = m$ 。一句话直觉: 逆元把乘法“倒回去”—— e 把 m 幂了一次, d 正好把这次幂抵消, 于是 c^d 回到 m 。前提仍是 $\gcd(e, \varphi(n)) = 1$, 否则 d 根本不存在。

私钥为什么不能公开: 知道 d 就能解密一切。而要算 d 就得知道 $\varphi(n)$, 就得分解 n 。所以“ n 很大且难分解”是 RSA 安全的根基。反过来, 题目给的 n 要么太小、要么被重复使用, 根基就塌了——这正是 C3 和 C4 的突破口。

逐字节加密还是整体加密 要特别检查消息是怎样变成整数的。本地 C3 **逐字节加密**：每个明文字节单独做一次模幂，所以每个密文整数对应一个字节，解出的整数应在 0–255。C4 则把**整个字节串按大端序转换成一个大整数**再做一次模幂，解密后要用正确的字节序还原整个字节串。两者解密后的还原方式不同。Python 的 `int.from_bytes` 与 `int.to_bytes` 可以控制字节序（“大端” = 高位字节在前）；长度未知时须根据题面或文件格式推断，不能随便丢掉开头的零字节。

一句话记忆

RSA 题先找 (n, e, c) 三元组，再问两个问题： n 能不能分解？ n 是不是被用了不止一次？

2.6 小模数为什么危险：试除分解

当 n 很小时（比如 $n = 3233$ ），分解它只需要**试除**：从 2 开始逐个试 n 能不能被整除，试到 $\lfloor \sqrt{n} \rfloor$ 就够了——因为 $n = p \times q$ 的两个因子不可能都大于 $\sqrt{n}$ 。找到 p 就得到 $q = n/p$ ，接着算 $\varphi(n)$ 、求 d 、解密。

$n = 3233$ 时 $\sqrt{n} \approx 56.8$ ，试到 56 找到 53，另一个因子是 61。这就是 C3 的全部数学。真正的 RSA 用几百位甚至几千位的 n ，试除完全不可行；真实 RSA 还依赖正确填充等工程细节。不能把教学题的试除法当作普遍攻击。

常见误区

$\sqrt{n}$ 是试除的上限，不是随便试。超过 $\lfloor \sqrt{n} \rfloor$ 还没找到因子，说明 n 是素数——这本身就是重要信息。

2.7 共模攻击为什么成立：同一 n 、两个 e

如果同一段明文 m 被同一个模数 n 下的两个不同指数 e_1, e_2 加密，得到 $c_1 \equiv m^{e_1}, c_2 \equiv m^{e_2} \pmod{n}$ ，并且 $\gcd(e_1, e_2) = 1$ ，那么可以不分解 n 就恢复 m 。原理是数论里的**扩展欧几里得算法**：能找到整数 a, b 使

$$e_1 a + e_2 b = 1.$$

于是

$$c_1^a \cdot c_2^b \equiv m^{e_1 a + e_2 b} \equiv m^1 \equiv m \pmod{n}.$$

负的系数（比如 $a = -8$ ）表示取**模逆元**： c_1^{-8} 先求 c_1 的逆再乘 8 次幂。C4（玄机 587）正是这种情形：两条回执记录共用一个 308 位十进制的 n ，指数分别是 65537 与 17，且 $65537 \times (-8) + 17 \times 30841 = 1$ 。

那对系数 a, b 是怎么算出来的 上式的 a, b 由扩展欧几里得算法给出，它是“辗转相除”的加强版，分两步理解：

1. 普通辗转相除求 gcd: 反复用“大数除以小数、取余数, 再把除数当新的大数”, 直到余数为 0, 最后一个非零余数就是最大公约数。例如求 $\text{gcd}(65537, 17)$: $65537 = 3855 \times 17 + 2$, $17 = 8 \times 2 + 1$, $2 = 2 \times 1 + 0$, 所以 $\text{gcd}(65537, 17) = 1$ 。
2. 扩展版在回代时顺带记录每一步, 把余数写成两个原数的整数组合, 最后得到 $xa + yb = \text{gcd}(x, y)$ ——这条等式叫 **Bézout** (贝祖) 等式。

当 $\text{gcd}(e_1, e_2) = 1$ 时, Bézout 等式右边正好是 1, 代入前一个公式就把两个模幂拼成“指数为 1”的一次幂, 直接得到 m , 全程不碰 n 的分解。图 2.6 是真实运行: $\text{math.gcd}(12, 18) = 6$ (不互素, 逆元不存在) 与 $\text{math.gcd}(65537, 17) = 1$ 形成对照, 辗转相除只走了 3 步, 扩展版给出 $a = -8$ 、 $b = 30841$, 代回验证 $65537 \times (-8) + 17 \times 30841 = 1$ 。这也解释了为什么“指数互素”是共模攻击的前提: 不互素时 Bézout 右边是 $\text{gcd} > 1$, 就拼不出指数为 1 的 m 。

```

gcd and Extended Euclid (Bezout)

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -X utf8
figures/raw/fig-crypto-24-eea-demo.py
=== 1. gcd: 能同时整除两数的最大正整数, =1 叫“互素” ===
math.gcd(12, 18) = 6 -> 不互素, 逆元不存在
math.gcd(65537, 17) = 1 -> 互素, 逆元存在
=== 2. 辗转相除: 反复“大数除以小数取余”, 直到余数为 0 ===
求 gcd(65537, 17) 的每一步:
    65537 = 3855 * 17 + 2
    17 = 8 * 2 + 1
    2 = 2 * 1 + 0
    gcd(65537, 17) = 1
=== 3. 扩展欧几里得: 顺带求出 Bézout 系数 a, b, 使 x*a + y*b = g ===
扩展欧几里得(65537, 17) -> g = 1, a = -8, b = 30841
验证 65537*-8 + 17*30841 = 1
=== 4. 逆元: g = 1 时, Bézout 系数就是模逆元 ===
17*30841 = 1 (mod 65537) -> 17 模 65537 的逆元是 30841
RSA 里同一个道理: pow(17, -1, 3120) = 2753
验证 (17*2753) % 3120 = 1

```

图 2.6: 第 2 章 gcd、辗转相除与扩展欧几里得: 由 Bézout 系数 $a = -8$ 、 $b = 30841$ 得 $65537a + 17b = 1$, 并演示逆元 (真实终端实录)

一句话记忆

同一明文 + 同一模数 + 两个互素指数 = 不用分解 n 也能解密。

2.8 自定义符号编码：五格电文的方法论

有些题不用标准编码，而是给一套自定义符号表。玄机 586 “五格电文”的附件给出 32 个互不相同的符号、一份公开试报和一段待译正文。方法论只有一句话：先猜表示方式，再用已知样本验证。

1. **数一数符号种类**：32 个符号，而 $2^5 = 32$ ，最自然的假设是“每个符号表示 5 位”；
2. **找已知明文**：附件给了公开试报，明文是“夜班 OK”；
3. **建立映射并验证**：每个符号换成它在字模板中的下标、写成 5 位二进制，从左到右每 8 位拼成一个字节。用试报做实验，能解出“夜班 OK”，说明符号顺序、位序、字节编码都猜对了；
4. **同法处理正文**：映射验证通过后，把完全相同的变换用于待译正文。

注意 5 位符号的边界和 8 位字节的边界**不对齐**，所以必须先把符号换成比特串、拼接、再按 8 位重新分组，见图 2.7。抄报员加的空格只是分组记号，处理时一律去掉。

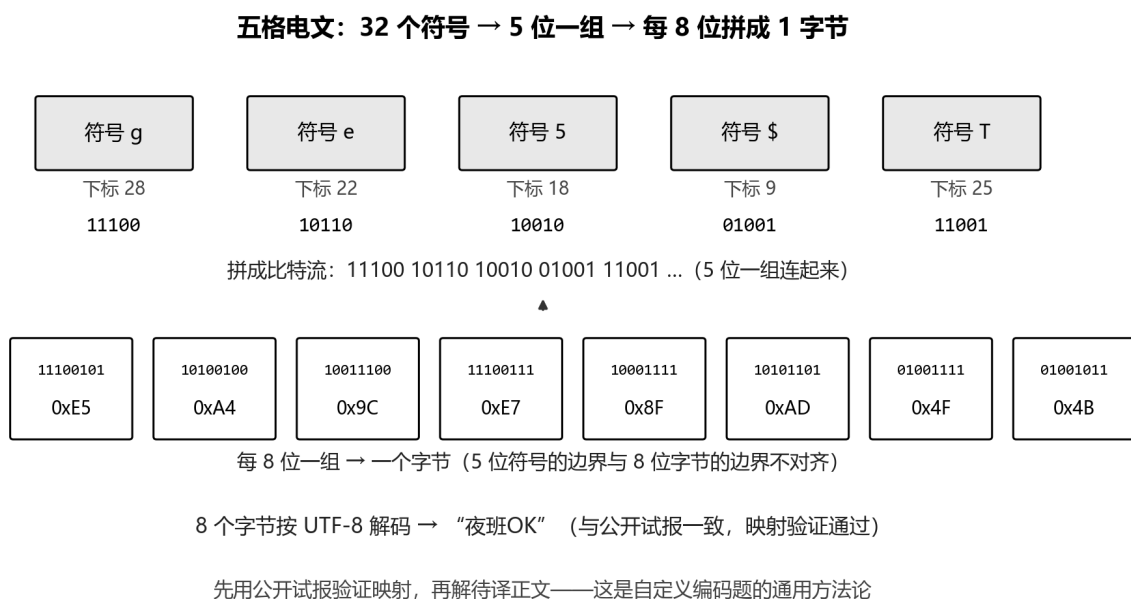


图 2.7: 第 2 章五格电文比特流示意：符号 → 下标 → 5 位 → 拼接 → 8 位一格 → UTF-8 解码出“夜班 OK”（原理示意图，数据来自 586 真实附件）

常见误区

看到 32 个符号不要直接套标准 Base32。标准 Base32 的字母表、填充规则都是固定的，自定义编码必须用自己的字模板和已知样本验证，不能假设“和 Base32 一样”。

2.9 本章知识点速查

知识点	一句话要点
字符/字节/十六进制	字符给人看，字节给计算机存，十六进制给眼睛看字节； <code>b'41'</code> 有 2 字节， <code>fromhex('41')</code> 只有 1 字节。
编码 vs 加密	Base64、十六进制都是编码：规则公开、无密钥；加密有密钥。
异或	$(x \oplus k) \oplus k = x$ ；单字节密钥只有 256 种，可穷举。
可读性判断	看前缀、字符范围、语言、结构四条一起看，别只看一个字符。
模运算	除以 n 取余； <code>pow(b, e, n)</code> 快速幂； <code>pow(a, -1, n)</code> 求逆元。
RSA 三元组	(n, e, c) 公开； d 私有， $ed \equiv 1 \pmod{\varphi(n)}$ ， $n = pq$ 。
小模数	n 小到 $\lfloor \sqrt{n} \rfloor$ 能试完，试除即可分解。
共模	同 n 双 e 且互素，用 Bézout 系数组合出明文。
自定义编码	先猜表示方式，再用已知样本验证；5 位符号与 8 位字节边界不对齐。

第三章 Crypto 工具箱

本章目标：三个主力工具上手：python 的一行命令与脚本、xxd 看十六进制、pow 三参数做快速幂与求逆元。

本章每个工具都按“是什么 / 怎么装 / 基本用法 / 真实输出”介绍。工具不多，但每一个都会在第 4 章反复出现。

3.1 python3：交互、一行命令与脚本

是什么 Python 是本册唯一的“编程工具”。解题的三件事——读文件、做变换、打印核对量——都靠它。3.8 以上版本即可，标准库就够，不用装第三方包。

怎么装 Windows 在 <https://www.python.org/downloads/> 下载安装包，安装时勾选 Add python.exe to PATH。WSL 里执行 `sudo apt install python3`。装完用 `python --version` 验证。

基本用法 三种形态，按需要选：

- **一行命令：**`python -c '...'` 适合两三行就说完的小实验；
- **脚本文件：**把代码存成 `xxx.py` 再 `python xxx.py`，适合超过三行、要反复修改的解题代码；
- **交互模式：**直接敲 `python` 进入 `>>>` 提示符，适合随手验证一个表达式（输 `exit()` 离开）。

图 3.1 是真实执行的三条命令：查版本、一行命令循环打印、导入标准库算最大公约数。

```
python one-liners and stdlib

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python --version
Python 3.12.10
PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'for i in range(3):
print('第', i, '次打印')'
第 0 次打印
第 1 次打印
第 2 次打印
PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'import sys, math;
print(sys.version); print(math.gcd(12, 18))'
3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)]
6
```

图 3.1: 第 3 章 python 基本用法：版本确认、-c 一行命令、标准库随手用（真实终端实录）

PowerShell 引号规则（看懂截图里的命令）

截图里的一行命令写成 `python -c 'print('hello')'`，为什么单引号都是两个？因为 PowerShell 的单引号字符串里，一个单引号要写两遍才表示一个单引号字符；外层单引号把整段代码括成一个参数传给 python。所以 `'hello'` 到 python 手里就是 `'hello'`。照抄截图里的命令就能得到同样的输出。

常见误区

一行命令超过三四行就写成脚本文件。一行命令里改一个逗号都容易改错，脚本文件可以用编辑器检查、反复运行。

3.2 xxd：看文件的十六进制视图

是什么 xxd 把文件的原始字节按十六进制打印，右侧还带 ASCII 预览列。它是“看清文件到底存了什么字节”的最直接工具。在 WSL/Ubuntu 里自带（`sudo apt install xxd` 可补装），Windows 侧可用 WSL 运行。

基本用法

```
xxd 文件名          # 完整十六进制视图
xxd 文件名 | head    # 只看前若干行（WSL/bash）
```

图 3.2 是真实执行：对 `labs/crypto/xor.hex` 运行 `xxd`，注意左侧十六进制和右侧 ASCII 列——右侧显示的是 `515b5650...` 这些字符，说明这个文件里存的是十六进制文本，不是密文字节本身；对 `rsa.txt` 运行则看到 `n=3233...` 的明文参数。这一眼就能把“文件内容”和“文件显示”分开。

```

xxd: hexadecimal view of a file

mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && xxd
labs/crypto/xor.hex
00000000: 3531 3562 3536 3530 3463 3466 3538 3435  515b56504c4f5845
00000010: 3638 3565 3434 3638 3539 3538 3433 3638  685e446859584368
00000020: 3561 3536 3530 3565 3534 3461 0d0a      5a56505e544a..
mzj@Ubuntu-24.04:~$ cd /mnt/c/Users/mzj/Desktop/CTF/CTF-training-problem-first-full && xxd
labs/crypto/rsa.txt | head -6
00000000: 6e3d 3332 3333 0d0a 653d 3137 0d0a 6369  n=3233..e=17..ci
00000010: 7068 6572 7465 7874 3d5b 3133 3639 2c20  phertext=[1369,
00000020: 3734 352c 2031 3633 322c 2032 3932 332c  745, 1632, 2923,
00000030: 2038 3535 2c20 3234 3132 2c20 3132 3330  855, 2412, 1230
00000040: 2c20 3136 3332 2c20 3131 392c 2031 3233  , 1632, 119, 123
00000050: 302c 2032 3237 312c 2031 3633 322c 2037  0, 2271, 1632, 7

```

图 3.2: 第 3 章 xxd 查看文件: xor.hex 存的是十六进制文本, rsa.txt 存的是可读参数 (真实终端实录, WSL)

一句话记忆

xxd 右侧 ASCII 列有可读字符 = 这个文件存的是文本; 全是一堆随机符号才是原始二进制字节。

3.3 pow 三参数: 快速幂与求逆元

是什么 Python 内置的 `pow(b, e, n)` 计算 $b^e \bmod n$, 先做快速幂再取模, 数值多大都不怕; `pow(a, -1, n)` 计算 a 模 n 的逆元 (Python 3.8 起支持)。这两个是 RSA 题的日常操作。

基本用法 见图 3.3 的真实执行:

```

pow(2, 10, 1000)      # 2^10 = 1024, 模 1000 得 24
pow(17, -1, 3120)     # 17 模 3120 的逆元 = 2753
(17 * 2753) % 3120    # 验证乘回去余 1
math.gcd(65537, 17)   # 判断两个指数是否互素

```

注意 `pow(2, 10)` 两参数版本才是普通乘方 (得 1024); 三参数版本才是模幂。RSA 解密就是一句 `pow(c, d, n)`。

```

pow() fast modpow and modular inverse

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'print("pow(2,10,1000) 快速幂
= ', pow(2,10,1000)); print("pow(17,-1,3120) 求逆元 = ', pow(17,-1,3120)); print("验证 (17*2753) %
3120 = ', (17*2753) % 3120)'
pow(2,10,1000) 快速幂 = 24
pow(17,-1,3120) 求逆元 = 2753
验证 (17*2753) % 3120 = 1
PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'import math;
print("math.gcd(65537,17) = ', math.gcd(65537,17)); print("pow(2,10) 不取模 = ', pow(2,10))'
math.gcd(65537,17) = 1
pow(2,10) 不取模 = 1024

```

图 3.3: 第 3 章 pow 用法: 快速幂、求逆元、验证逆元、判断互素 (真实终端实录)

求逆元的前提 a 与 n 必须互素 (最大公约数为 1), 否则逆元不存在, $\text{pow}(a, -1, n)$ 会报 `ValueError`。RSA 里要求 $\text{gcd}(e, \varphi(n)) = 1$ 就是这个道理。

3.4 读附件的三件小工具

zipfile: 读压缩包 平台题附件是 ZIP。不用解压也能读:

```

from zipfile import ZipFile
z = ZipFile('labs/platform/attachments/586-five-grid.zip')
print(*z.namelist(), sep='\n')          # 列出包内文件
data = z.read('attachments/archive.json') # 读出文件内容 (字节)

```

json: 读结构化数据 附件里的 `archive.json`、`export.json` 是 JSON 格式 (键值对组成的文本):

```

import json
record = json.loads(data)          # 字节或文本 → Python 字典
print(record['glyph_plate'])       # 按键取值

```

ast: 读 Python 风格的数据列表 `rsa.txt` 里的密文写成 `ciphertext=[1369, 745, ...]`, 可以直接解析成列表:

```

import ast
text = open('labs/crypto/rsa.txt').read()
data = ast.literal_eval(text.split('ciphertext=', 1)[1].strip())

```

`ast.literal_eval` 只解析字面量（数字、列表、字符串），不会把数据当任意 Python 代码执行，比 `eval` 安全。

3.5 工具箱小结

想做什么	用什么	关键点
跑一行小实验	<code>python -c '...'</code>	PowerShell 里单引号写两遍表示一个单引号
把字节显示成十六进制	<code>xxd</code> 文件（WSL）	看 ASCII 列判断文件存的是文本还是字节
算 $b^e \bmod n$	<code>pow(b, e, n)</code>	快速幂，不怕大数
求 $a^{-1} \bmod n$	<code>pow(a, -1, n)</code>	需要 a 与 n 互素
十六进制转字节	<code>bytes.fromhex(s)</code>	长度必须是偶数
字节转十六进制	<code>b.hex()</code>	1 字节变 2 个字符
字节串转整数	<code>int.from_bytes(b, 'big')</code>	记得字节序
读 ZIP/JSON/listing	<code>zipfile</code> / <code>json</code> / <code>ast.literal_eval</code>	都是标准库

第四章 题目：题面卡与逐题图解带做

本章目标：五道题，每题一节：先用题面卡把“给什么、要什么、考什么”读清楚，紧接着就是这道题的完整解题步骤——读题与思路、分步操作与观察、结果与核对、为什么会成功、排错指南、变式练习。建议先只读题面卡自己动手试 10–15 分钟，卡住再看后面的步骤。

每张卡的字段固定为：题目来源 / 给定材料 / 目标 / 交付物 / 考点 / 前置知识 / 难度。难度用星表示：★☆☆入门、★★☆进阶、★★★★挑战。读完卡片先自己想两分钟，再往下看这道题的解题步骤。

做题顺序建议 C0 → C1 → C3 → C2 → C4。C0、C1 练表示与穷举，C3 练 RSA 基本操作，C2 练自定义编码的验证方法，C4 练数论关系。卡住超过 20 分钟就看第 4 章对应小节的“读题与思路”。

4.1 题 C0：十六进制字符串到底有多少字节？

题 C0 题面卡

题目来源：本地练习（离线可做），题面即本卡

给定材料：一段文本 666c61677b6865787d

目标：判断它的字符数、按十六进制还原后的字节数，并读出最终 ASCII 内容

交付物：一个 `flag{...}` 字符串；提交前核对字符数与花括号

考点：数据表示：十六进制是字节的显示方式，不是加密

前置知识：2.1 节字符/字节/十六进制；3.1 节 python 一行命令

难度：★☆☆（入门）

读题与思路 交付物是 flag 字符串，给的是一串十六进制字符。看到这种材料，第一反应不是“解密”，而是“换表示”：十六进制通常只是字节的显示方式。所以要问三个问题：现在手里的是文本还是字节？按两字符一字节还原后有多长？还原出的字节能不能按 ASCII 读出文字？题面还特意问“字符数、字节数、ASCII 内容”，这就是在提醒你别把字符数当字节数。

分步操作与观察

1. 在 PowerShell 里让 python 把字符串按十六进制还原，并打印字符数与字节数。命令（单引号内的单引号写两遍，见 3.1 节）：

```
python -c 's=\'666c61677b6865787d\'; print(\'文本长度(字符):\', len(s));  
b=bytes.fromhex(s); print(\'还原后字节数:\', len(b)); print(\'字节内容:\', b);  
print(\'ASCII 解码:\', b.decode(\'ascii\'))'
```

图 4.1 是真实执行结果。应该看到：文本长度 18、还原后字节数 9、字节内容显示成 `b'flag{hex}'`。这一步是把“十六进制文本”换回“字节”。

2. 对比两种“41”，确认自己没有把显示方式当内容：

```
python -c 'b1=b\'41\'; b2=bytes.fromhex(\'41\');  
print(\'两个字符的文本 b41:\', b1, \'长度\', len(b1));  
print(\'解析成 1 个字节后:\', b2, \'长度\', len(b2))'
```

同一张图 4.1 的第二条命令就是它：`b'41'` 长度 2，`b'A'` 长度 1。应该看到两个长度不同。

3. 用 `.decode('ascii')` 把 9 个字节读成文本。ASCII 只定义了 0-127 的字符，flag 里的字母、花括号都在其中，能无损读出。



```
C0: 18 hex chars = 9 bytes  
  
PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 's=\'666c61677b6865787d\';  
print(\'文本长度(字符):\', len(s)); b=bytes.fromhex(s); print(\'还原后字节数:\', len(b)); print(\'字节内容:\', b);  
print(\'ASCII 解码:\', b.decode(\'ascii\'))'  
文本长度(字符): 18  
还原后字节数: 9  
字节内容: b'flag{hex}'  
ASCII 解码: flag{hex}  
  
PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'b1=b\'41\';  
b2=bytes.fromhex(\'41\'); print(\'两个字符的文本 b41:\', b1, \'长度\', len(b1)); print(\'解析成 1 个字节后:\', b2,  
'长度', len(b2))'  
两个字符的文本 b41: b'41' 长度 2  
解析成 1 个字节后: b'A' 长度 1
```

图 4.1: 第 4 章 C0: 18 个十六进制字符还原成 9 个字节，再按 ASCII 读出（真实终端实录）

结果与核对 最终 ASCII 内容是 `flag{hex}`。独立核对：把还原出的字节重新编码回十六进制，应得到原字符串；字符数 $18 = 9 \times 2$ ，每字节两个十六进制字符，数量关系对得上。

```
flag: flag{hex}
```

为什么会成功 一句话机制：十六进制是字节的显示方式，不是加密。展开说：这道题没有任何密钥和算法，只有一次表示转换。18 个十六进制字符按“两个字符一组”解读，每组是 1 个

字节的值；9 个字节按 ASCII 表解读成 9 个字符。理解了这层，就能理解为什么 Crypto 题的第一步永远是“识别表示”。

排错指南

- `bytes.fromhex` 报 `non-hexadecimal number`：字符串里混进了空格、换行或非十六进制字符，先 `.strip()` 并检查长度是否为偶数；
- 长度是奇数：多复制或少复制了一个字符，回题面重新数；
- 解码出乱码：确认用的是 `decode('ascii')` 或 `decode('utf-8')`，且数据没有被当作十六进制处理两次。

变式练习

1. 把 666c6167 还原成字节并读出文字，记录每一步的类型与长度（提示：4 个字符 → 4 个字节？先数清楚）。
2. 给字符串 `flag{hex}` 用 `.encode().hex()` 编码回十六进制，核对是否得到原题数据。
3. 找一段 Base64 文本（如 `ZmxhZ3toZXh9`）用 `base64.b64decode` 解码，记录“字符数 → 字节数”的关系，并对比本题的十六进制。

4.2 题 C1：单字节异或如何从密文找回明文？

题 C1 题面卡

题目来源：本地靶场 `labs/crypto/xor.hex`（离线可做）

给定材料：密文文件 `labs/crypto/xor.hex`；题目说明：密钥只有 1 个字节，循环作用于每个明文字节

目标：找出密钥，还原明文

交付物：密钥（十六进制）与一个 `flag{...}` 字符串；两样都要写出核对过程

考点：异或的可逆性；256 种密钥的穷举与可读性判断

前置知识：2.2 节异或真值表、2.3 节单字节穷举；3.2 节 `xxd`（可选）

难度：★☆☆（入门）

读题与思路 题目说密钥只有 1 个字节、循环作用于每个明文字节。这句翻译成搜索量就是：**256 种候选**。256 种完全试得起，所以本题的正路是穷举 + 可读性筛选，不是猜。动手前先看材料本身：`labs/crypto/xor.hex` 是什么表示？文件名已经提示是十六进制。用 `Get-Content` 看它的内容（或 `xxd` 看它的字节），确认之后再写代码。筛选真明文的线索是题面格式：`flag` 通常以 `flag{` 开头且全是可打印字符。

分步操作与观察

1. 先看密文文件内容，再转成字节：

```
Get-Content labs/crypto/xor.hex
python -c 't=open(''labs/crypto/xor.hex'').read().strip();
print(''十六进制字符数:'', len(t)); b=bytes.fromhex(t);
print(''字节数:'', len(b)); print(''前 8 字节的十六进制:'', b[:8].hex());
print(''直接打印字节串(乱码正常):'', b)'
```

图 4.2 是真实执行结果。应该看到：44 个十六进制字符、22 个字节。直接打印字节串会看到 Q[VPLOXeh^DYXChZVP^TJ 之类的乱码——这正是异或后的密文“不可读”的直观样子。（注意 `.strip()` 是为了去掉文件末尾的换行，不加会多出一个字节。）

2. 穷举 256 个密钥并观察候选：

```
python -c 'data=bytes.fromhex(open(''labs/crypto/xor.hex'').read().strip());
plain=lambda k: bytes(x^k for x in data);
[print(hex(k), '->', plain(k).decode(''ascii'', errors='''replace'''))
 for k in range(0x33, 0x3c)];
hit=[k for k in range(256)
     if plain(k).startswith(b''flag{'') and all(32<=c<127 for c in plain(k))];
print(''--- 256 个候选全部试完，全可读且以 flag{ 开头的密钥:'', [hex(k) for k in
hit]);
print(''密钥'', hex(hit[0]), '明文'', plain(hit[0]).decode())'
```

图 4.3 是真实执行结果。打印 0x33 到 0x3b 这一段可以看到“差一点”的候选全是乱码，只有 0x37 那一行是 `flag{xor_is_not_magic}`；最后的筛选行显示 256 个候选中只有 '0x37' 满足“全部可打印 + 以 `flag{` 开头”。这一步是穷举与可读性判断。

3. 用两个独立方法核对密钥（图 4.4）：

```
python -c 'data=bytes.fromhex(open(''labs/crypto/xor.hex'').read().strip());
k=data[0]^ord(''f''); print(''由首字节 f 反推密钥:'', hex(k));
plain=bytes(x^k for x in data); print(''明文:'', plain.decode());
back=bytes(x^k for x in plain);
print(''明文再异或得到的十六进制:'', back.hex());
print(''与原密文一致:', back.hex()==open(''labs/crypto/xor.hex'').read().strip
())'
```

应该看到：首字节法也得到 0x37；明文再异或一遍得到的十六进制与原密文逐字符一致，打印 True。

```
C1: read xor.hex and convert to bytes

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> Get-Content labs/crypto/xor.hex
515b56504c4f5845685e4468595843685a56505e544a
PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c
't=open('labs/crypto/xor.hex').read().strip(); print('十六进制字符数:', len(t)); b=bytes.fromhex(t);
print('字节数:', len(b)); print('前 8 字节的十六进制:', b[:8].hex()); print('直接打印字节串(乱码正常):', b)'
十六进制字符数: 44
字节数: 22
前 8 字节的十六进制: 515b56504c4f5845
直接打印字节串(乱码正常): b'Q[VPLOxEh^DhYXChZVP^TJ'
```

图 4.2: 第 4 章 C1 第 1 步: xor.hex 是十六进制文本, 44 个字符换 22 个字节 (真实终端实录)

```
C1: brute-force all 256 one-byte keys

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c
'data=bytes.fromhex(open('labs/crypto/xor.hex').read().strip()); plain=lambda k: bytes(x^k for x
in data); [print(hex(k), '->', plain(k).decode('ascii', errors='replace')) for k in
range(0x33, 0x3c)]; hit=[k for k in range(256) if plain(k).startswith(b'flag{') and all(32<=c<127
for c in plain(k))]; print('--- 256 个候选全部试完, 全可读且以 flag{ 开头的密钥:', [hex(k) for k in hit]);
print('密钥', hex(hit[0]), '明文', plain(hit[0]).decode())'
0x33 -> bhec|kv[mw[jkp[iecmgy
0x34 -> eobdx{lq\jp\mlw\nbdj`~
0x35 -> dnceyzmp]kq]lmv]oceka@
0x36 -> gm`fzyns^hr^onu^l`fhb|
0x37 -> flag{xor_is_not_magic}
0x38 -> icnhtw`}Pf|Pa`{Pbnhf1r
0x39 -> hboiuva|Qg}Q`azQcoigms
0x3a -> ka1jvub@Rd~RcbyR`ljdnP
0x3b -> j`mkwtc~Se@SbcxSamkeoq
--- 256 个候选全部试完, 全可读且以 flag{ 开头的密钥: ['0x37']
密钥 0x37 明文 flag{xor_is_not_magic}
```

图 4.3: 第 4 章 C1 第 2 步: 256 个单字节密钥穷举, 只有 0x37 命中可读 flag (真实终端实录)

```
C1: key from first byte + re-XOR check

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c
'data=bytes.fromhex(open('labs/crypto/xor.hex').read().strip()); k=data[0]^ord('f');
print('由首字节 f 反推密钥:', hex(k)); plain=bytes(x^k for x in data); print('明文:', plain.decode());
back=bytes(x^k for x in plain); print('明文再异或得到的十六进制:', back.hex()); print('与原密文一致:',
back.hex()==open('labs/crypto/xor.hex').read().strip())'
由首字节 f 反推密钥: 0x37
明文: flag{xor_is_not_magic}
明文再异或得到的十六进制: 515b56504c4f5845685e4468595843685a56505e544a
与原密文一致: True
```

图 4.4: 第 4 章 C1 第 3 步: 首字节反推密钥 0x37, 反向异或与原密文一致 (真实终端实录)

结果与核对 密钥是 0x37 (十进制 55), 明文是 `flag{xor_is_not_magic}`。三重独立核对: 穷举筛选唯一命中、首字节公式 $k = c_0 \oplus \text{ord}(f)$ 同样得到 0x37、明文反向异或还原出原密文。

flag: `flag{xor_is_not_magic}`

为什么会成功 一句话机制: $(x \oplus k) \oplus k = x$, 且单字节密钥空间只有 256。展开说: 异或加密把明文每个字节和同一个 k 做异或; 解密再做一次就还原。因为 k 只有 256 种取值, 攻击者可以全部试一遍, 用“可打印 + `flag{` 前缀”把假候选排掉。首字节法是同一原理的捷径: 已知明文第一个字节是 f , 就能直接算出 k 。但请注意: 这种方法只在密钥空间小或重复使用时成立; 若密钥与消息等长、真正随机、只用一次, 穷举就没有意义。

排错指南

- `bytes.fromhex` 报奇数长度: 多半是读文件时带了换行, 先 `.strip()`;
- 穷举结果全是乱码、没有一个像 `flag:` 确认异或对象是“字节”而不是十六进制“文本”; 别对 `t` 直接异或, 要先 `bytes.fromhex(t)`;
- 筛选出多个候选: 加强筛选条件, 检查完整 `flag{` 前缀、花括号配对和字符范围, 短数据尤其容易误判;
- `decode('ascii')` 报错: 加 `errors='replace'` 先看内容, 或改用 `errors='ignore'` 并记录丢掉了什么。

变式练习

1. 自己写一段 `flag{my_name}` 的明文, 用密钥 0x42 异或加密, 把密文十六进制交给同学 (或隔一天的自己) 用穷举解回。
2. 把题目改成 3 字节循环密钥: 位置 0,3,6,... 共用 k_0 , 位置 1,4,7,... 共用 k_1 , 位置

2, 5, 8, ... 共用 k_2 。提示：按下标对 3 取余分组，每组就是一道单字节异或题。解完用反向异或核对整段。

- 只用首字节法解本题 ($k = c_0 \oplus \text{ord}(f)$)，不跑穷举。再故意假设首字节是 F（大写）重算，观察结果为何不可信，体会“已知明文假设也要验证”。

4.3 题 C2：五格电文的符号到底表示什么？（玄机 586）

题 C2 题面卡

题目来源：玄机平台 2026 安网杯-五格电文，<https://xj.edisec.net/challenges/586>;

附件本地副本 labs/platform/attachments/586-five-grid.zip（离线可做到解出 flag）

给定材料：附件 ZIP 内有字模板 glyph_plate（32 个互不相同的符号）、公开试报 sample_wire 与已知明文 sample_plain（“夜班 OK”）、待译正文 dispatch_wire，以及说明文件 format.txt（抄报员以空格断句，誊录时一律省略）

目标：推出符号到比特的映射，翻译正文，读出其中的 flag

交付物：一个 flag{...} 字符串（32 位十六进制样式），在玄机平台对应题目页提交

考点：自定义符号编码：先猜表示方式（32 符号 $\approx$ 5 位），再用已知样本验证映射与位序

前置知识：2.1 节编码与表示；2.8 节自定义编码方法论；3.4 节 zipfile/json

难度：★★☆（进阶）

读题与思路 材料是 32 个怪符号组成的电文、一份公开试报和一段待译正文。看到“32 个符号”，第一个假设应该是**每个符号表示 5 位**，因为 $2^5 = 32$ 。但注意：这只是假设。验证假设的钥匙在附件里——公开试报的明文已知（“夜班 OK”），它是你的**标尺**。所以正确的动作顺序是：数符号 $\rightarrow$ 提假设 $\rightarrow$ 用试报验证 $\rightarrow$ 验证通过才解正文。反过来“看到 32 个字符就套标准 Base32”是错的：标准 Base32 的字母表是固定的 A-Z2-7，而这里的符号是自定义的，必须用字模板自己建立映射。

分步操作与观察

1. 打开附件副本，先看包里有什么：

```
python -c 'from zipfile import ZipFile;
z=ZipFile('labs/platform/attachments/586-five-grid.zip');
print(*z.namelist(), sep='\n')
```

图 4.5 上半部分是真实执行结果：包里有 attachments/archive.json（数据）和 attachments/format.txt（说明）。说明文件写明：字模板上的符号次序与电文取值相关；抄报员以空格断句，仅为换气记号，誊录时一律省略。

2. 读出数据并核对符号表的前提条件:

```
python -c 'from zipfile import ZipFile; import json;
r=json.loads(ZipFile(''labs/platform/attachments/586-five-grid.zip'')
              .read(''attachments/archive.json''));
print(''archive.json 的字段:'', sorted(r));
print(''glyph_plate:'', r[''glyph_plate'']);
print(''符号数:'', len(r[''glyph_plate'']), '', 去重后:'', len(set(r[''
    glyph_plate''])))');
print(''sample_wire:'', r[''sample_wire'']);
print(''dispatch_wire 前 80 字符:'', r[''dispatch_wire''][:80])'
```

图 4.5 下半部分是真实执行结果。应该看到：字段有 dispatch_wire、glyph_plate、sample_plain、sample_wire、station；符号数 32、去重后还是 32。**互不重复**很关键——它保证“符号 → 下标”的映射是唯一确定的。

3. 用公开试报验证映射（本题的核心一步）:

```
python -c 'from zipfile import ZipFile; import json;
r=json.loads(ZipFile(''labs/platform/attachments/586-five-grid.zip'')
              .read(''attachments/archive.json''));
a=r[''glyph_plate'']; w=r[''sample_wire''].replace('' ', '');
bits=''.join(f''{a.index(s):05b}'' for s in w);
print(''试报符号数:'', len(w), '', 比特数:'', len(bits));
print(''前 12 个符号在模板中的下标:'', [a.index(s) for s in w[:12]]);
print(''拼出的比特串前 40 位:'', bits[:40]);
b=bytes(int(bits[i:i+8], 2) for i in range(0, len(bits)//8*8, 8));
print(''每 8 位组装成字节:'', b); txt=b.decode(''utf-8'');
print(''UTF-8 解码结果:'', txt);
print(''与 sample_plain 一致:'', txt==r[''sample_plain''])'
```

图 4.6 是真实执行结果。注意看四件事：去掉空格后 13 个符号 → 65 个比特；每个符号换成字模板下标再写成 5 位二进制；每 8 位拼成一个字节（b'\xe5\xa4\x9c...'）；UTF-8 解码得到 夜班 OK，与 sample_plain 比对打印 True。映射、位序、字节编码三项同时验证通过。

4. 同一个变换处理正文并输出 flag:

```
python labs/platform/solve_five_grid.py
python labs/platform/solve_five_grid.py labs/platform/attachments/586-five-grid
.zip
```

图 4.7 是真实执行结果（两条命令分别是“默认读本地附件”和“显式给 ZIP 路径”）。脚本会先自动核对公开试报再输出正文，你应该看到 sample: 夜班 OK、完整正文和 flag: flag{c619...}。

5. 到玄机平台题目页 <https://xj.edisec.net/challenges/586> 提交 flag，等待提交按钮结束加载，确认页面显示“FLAG 正确”。

```
C2: 586 attachment layout and glyph plate

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'from zipfile import
ZipFile; z=ZipFile(''labs/platform/attachments/586-five-grid.zip''); print(*z.namelist(),
sep=chr(10))'
attachments/
attachments/archive.json
attachments/format.txt
PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'from zipfile import
ZipFile; import json; r=json.loads(ZipFile(''labs/platform/attachments/586-five-grid.zip'').read(''a
ttachments/archive.json'')); print(''archive.json 的字段:'', sorted(r)); print(''glyph_plate:'',
r[''glyph_plate'']); print(''符号数:'', len(r[''glyph_plate'']), '', 去重后:'',
len(set(r[''glyph_plate'']))); print(''sample_wire:'', r[''sample_wire'']); print(''dispatch_wire 前
80 字符:'', r[''dispatch_wire''][:80])'
archive.json 的字段: ['dispatch_wire', 'glyph_plate', 'sample_plain', 'sample_wire', 'station']
glyph_plate: Af#XwWs%Z$QYcqDJ085bMre4kTp9gCB7
符号数: 32 , 去重后: 32
sample_wire: ge5$TTgJ rr%Me
dispatch_wire 前 80 字符: ge5$TTgJ r4MQCc%W 4eQBYqC5 gpDY8TqJ 4%8kfA4$ D#qd8Zcc gpDZJpWd eADsc9Xf
cCCesq88
```

图 4.5: 第 4 章 C2 第 1–2 步: 586 附件结构、字模板 32 个符号互不重复、试报与正文（真实终端实录）

```
C2: verify mapping with known sample

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'from zipfile import
ZipFile; import json; r=json.loads(ZipFile(''labs/platform/attachments/586-five-grid.zip'').read(''a
ttachments/archive.json'')); a=r[''glyph_plate'']; w=r[''sample_wire''].replace('' ',''');
bits=''.join(f''{a.index(s):05b}'' for s in w); print(''试报符号数:'', len(w), '', 比特数:'', len(bits));
print(''前 12 个符号在模板中的下标:'', [a.index(s) for s in w[:12]]); print(''拼出的比特串前 40 位:'', bits[:40]);
b=bytes(int(bits[i:i+8], 2) for i in range(0, len(bits)//8*8, 8)); print(''每 8 位组装成字节:'', b);
txt=b.decode(''utf-8''); print(''UTF-8 解码结果:'', txt); print(''与 sample_plain 一致:'',
txt==r[''sample_plain''])'
试报符号数: 13 , 比特数: 65
前 12 个符号在模板中的下标: [28, 22, 18, 9, 25, 25, 28, 15, 21, 21, 7, 20]
拼出的比特串前 40 位: 1110010110100100100111001110011110001111
每 8 位组装成字节: b'\xe5\xa4\x9c\xe7\x8f\xadOK'
UTF-8 解码结果: 夜班OK
与 sample_plain 一致: True
```

图 4.6: 第 4 章 C2 第 3 步: 公开试报解出“夜班 OK”，与 sample_plain 一致 True（真实终端实录）

```
C2: run solve_five_grid.py -> flag

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python labs/platform/solve_five_grid.py
sample: 夜班OK
message: 夜班记录已核对。通行标记 flag{c619020e518c599051efc804f5f84884}
flag: flag{c619020e518c599051efc804f5f84884}
PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python
labs/platform/solve_five_grid.py labs/platform/attachments/586-five-grid.zip
sample: 夜班OK
message: 夜班记录已核对。通行标记 flag{c619020e518c599051efc804f5f84884}
flag: flag{c619020e518c599051efc804f5f84884}
```

图 4.7: 第 4 章 C2 第 4 步: solve_five_grid.py 输出正文与 flag（真实终端实录）

结果与核对 正文末尾的通行标记是：

flag: flag{c619020e518c599051efc804f5f84884}

三重核对：公开试报解出“夜班 OK”验证了映射；正文里 flag 符合 32 位十六进制样式；本账号此前在平台提交后显示正确反馈。样本报错时脚本会直接抛异常，不会输出正文——这就是“验证不过就不解正文”的自动化体现。

为什么会成功 一句话机制：已知明文是验证自定义编码假设的标尺。展开说：32 个符号给出“每符号 5 位”的候选假设；字模板给出“符号 → 数值”的映射；公开试报给出正确答案。把三者拼起来就能验证“下标顺序 + 位序 + 每 8 位分组 + UTF-8”整条链路，任何一环错了都

会在试报上露馅。链路验证通过后，正文只是同一条流水线的输入而已。另外注意 5 位边界和 8 位边界不对齐（图 2.7），所以必须先拼比特流再按 8 位重新分组，不能“两个符号凑一个字节”。

排错指南

- 试报解出来不对：先检查符号表顺序（是不是照抄字模板原文、有没有去重或排序）、再检查位序（下标写 5 位时高位在前）、最后检查是否去掉了所有空格；
- 解出的字节无法 UTF-8 解码：分组边界错了。打印比特串长度，确认按 $8k$ 位截断，余数位应丢弃（`len(bits)//8*8`）；
- 脚本报 `sample does not match`：映射没验证通过，正文先别解，回头查试报；
- 脚本报 `glyph_plate must contain 32 distinct symbols`：符号表读错了字段或数据损坏，重新解压附件。

变式练习

1. 把字模板倒序（或循环左移 3 位）生成一份新题，先故意用错映射解试报，观察报错；再改回正确映射解出正文。体会“样本验证”抓错的效率。
2. 不看脚本，手算 `sample_wire` 的前 3 个符号：写下它们在字模板中的下标、5 位二进制，和截图里的比特串前 40 位逐段对照。
3. 思考题：如果题目不给公开试报，只给正文，你还能验证映射吗？列出至少两个可以利用的弱假设（比如正文通常是 UTF-8 中文、结尾有 `flag{...}` 样式），并说明它们为什么只是弱假设。

4.4 题 C3：小模数 RSA 为什么能被试除？

题 C3 题面卡

题目来源：本地靶场 `labs/crypto/rsa.txt`（离线可做）

给定材料：参数文件 `labs/crypto/rsa.txt`： $n = 3233$ 、 $e = 17$ 、逐字节密文整数列表（23 项）

目标：恢复明文，并说清这道题弱在何处

交付物：一个 `flag{...}` 字符串 + 一段“弱点说明”（哪个数学条件没守住）

考点：RSA 三元组；小模数试除分解；求逆元与逐字节解密

前置知识：2.4 节模运算、2.5 节 RSA 三元组、2.6 节小模数；3.3 节 pow

难度：★★☆（进阶）

读题与思路 材料是三个数 $n = 3233$ 、 $e = 17$ 和 23 个密文整数。看到 RSA 先找三元组 (n, e, c) ，然后问两个问题（2.5 节的口诀）： n 能不能分解？ n 被用了几次？这里 $n = 3233$ 只有四位数， $\sqrt{n} \approx 56.8$ ，试除最多试 55 次就能分解——**可以分解**。再看题面说“逐字节密文列表”，意味着每个明文字节单独加密一次，解密出的每个整数应该都落在 0–255。所以路线是：分解 $n \rightarrow$ 算 $\varphi(n) \rightarrow$ 求 $d \rightarrow$ 逐项 $\text{pow}(c, d, n)$ 。

分步操作与观察

1. 读入参数并检查密文列表的合法性：

```
Get-Content labs/crypto/rsa.txt
python -c 'import ast;
t=open(''labs/crypto/rsa.txt'').read();
data=ast.literal_eval(t.split(''ciphertext='', 1)[1].strip());
print(''密文整数个数:'', len(data));
print(''最大值:'', max(data), '', 是否都小于 n=3233:'', all(c<3233 for c in data
));
print(''前 6 项:'', data[:6])'
```

图 4.8 是真实执行结果。应该看到 23 个整数、最大值 2923、True。每项都小于 n 是应该的：模幂的输出本就在 0 到 $n - 1$ 之间。

2. 试除分解 n 并求私钥 d ：

```
python -c 'import math; n=3233;
fs=[p for p in range(2, math.isqrt(n)+1) if n%p==0];
print(''从 2 试除到'', math.isqrt(n), '', 找到因子:'', fs, '', 另一因子:'', [n//
p for p in fs]);
phi=(61-1)*(53-1); d=pow(17, -1, phi);
print(''phi(n) ='', phi, '', 私钥指数 d ='', d);
print(''验证 (17*d) % phi ='', (17*d) % phi)'
```

图 4.9 上半部分是真实执行结果：试除到 56 找到因子 53，另一因子 61； $\varphi(n) = 3120$ ； $d = 2753$ ；验证 $(17 \times 2753) \bmod 3120 = 1$ 。这一步把“分解 $\rightarrow$ 欧拉函数 $\rightarrow$ 逆元”三件事串成一条链，每件都当场核对。

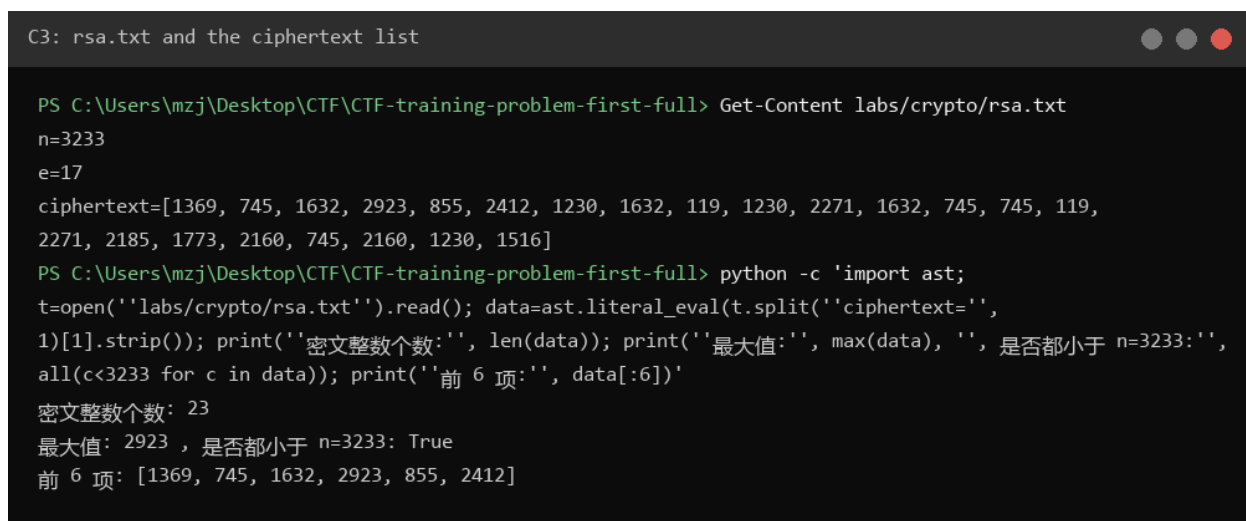
3. 用往返测试确认公式没错，再解密：

```
python -c 'n=3233; c=pow(65, 17, n);
print(''演示 pow(65,17,3233) ='', c, '', 再 pow(c,2753,3233) ='', pow(c, 2753,
n))'
```

同图 4.9 下半部分：随便取一个明文数 65，加密再解密回到 65，说明 (e, d) 配对正确。

```
python -c 'import ast;
t=open('labs/crypto/rsa.txt').read();
data=ast.literal_eval(t.split('ciphertext=', 1)[1].strip());
pt=bytes(pow(c, 2753, 3233) for c in data);
print('解密得到的前 6 个整数:', [pow(c, 2753, 3233) for c in data[:6]]);
print('明文:', pt.decode());
back=[pow(b, 17, 3233) for b in pt];
print('用公钥再加密与原密文一致:', back==data)'
```

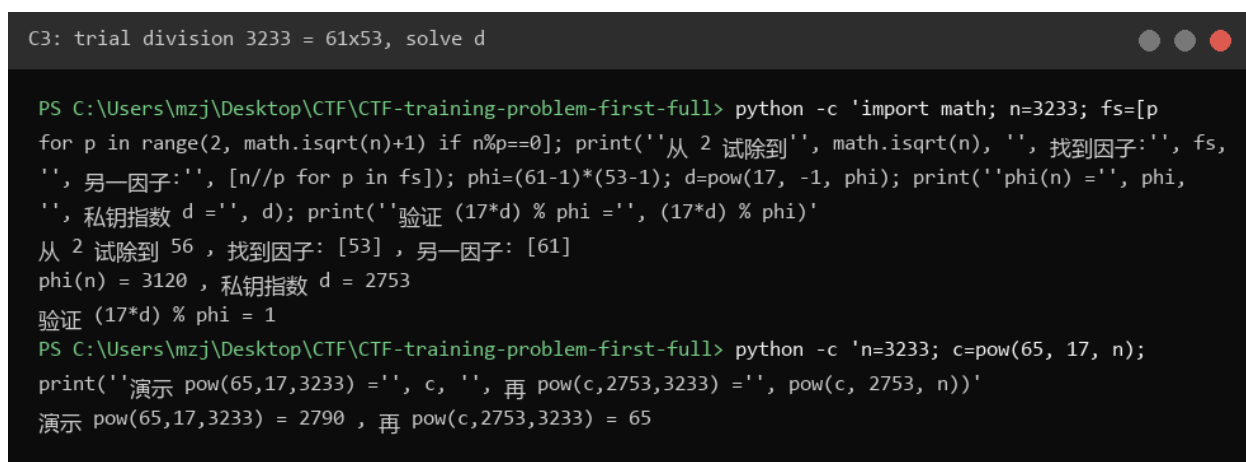
图 4.10 是真实执行结果。前 6 个整数是 102, 108, 97, 103, 123, 114，正是 f、l、a、g、{、r 的 ASCII 码；整段解码得到 flag；再加密比对打印 True。



```
C3: rsa.txt and the ciphertext list

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> Get-Content labs/crypto/rsa.txt
n=3233
e=17
ciphertext=[1369, 745, 1632, 2923, 855, 2412, 1230, 1632, 119, 1230, 2271, 1632, 745, 745, 119,
2271, 2185, 1773, 2160, 745, 2160, 1230, 1516]
PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'import ast;
t=open('labs/crypto/rsa.txt').read(); data=ast.literal_eval(t.split('ciphertext=',
1)[1].strip()); print('密文整数个数:', len(data)); print('最大值:', max(data), ', 是否都小于 n=3233:',
all(c<3233 for c in data)); print('前 6 项:', data[:6])'
密文整数个数: 23
最大值: 2923 , 是否都小于 n=3233: True
前 6 项: [1369, 745, 1632, 2923, 855, 2412]
```

图 4.8: 第 4 章 C3 第 1 步: rsa.txt 的参数与 23 项密文整数，全部小于 n（真实终端实录）



```
C3: trial division 3233 = 61x53, solve d

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'import math; n=3233; fs=[p
for p in range(2, math.isqrt(n)+1) if n%p==0]; print('从 2 试除到', math.isqrt(n), ', 找到因子:', fs,
', 另一因子:', [n//p for p in fs]); phi=(61-1)*(53-1); d=pow(17, -1, phi); print('phi(n) =', phi,
', 私钥指数 d =', d); print('验证 (17*d) % phi =', (17*d) % phi)'
从 2 试除到 56 , 找到因子: [53] , 另一因子: [61]
phi(n) = 3120 , 私钥指数 d = 2753
验证 (17*d) % phi = 1
PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'n=3233; c=pow(65, 17, n);
print('演示 pow(65,17,3233) =', c, ', 再 pow(c,2753,3233) =', pow(c, 2753, n))'
演示 pow(65,17,3233) = 2790 , 再 pow(c,2753,3233) = 65
```

图 4.9: 第 4 章 C3 第 2-3 步: 试除分解 $3233=61 \times 53$ 、求 $d=2753$ 、往返测试（真实终端实录）

```
C3: decrypt byte by byte -> flag

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'import ast;
t=open('labs/crypto/rsa.txt').read(); data=ast.literal_eval(t.split('ciphertext=',
1)[1].strip()); pt=bytes(pow(c, 2753, 3233) for c in data); print('解密得到的前 6 个整数:', [pow(c, 2753,
3233) for c in data[:6]]); print('明文:', pt.decode()); back=[pow(b, 17, 3233) for b in pt];
print('用公钥再加密与原文一致:', back==data)'
解密得到的前 6 个整数: [102, 108, 97, 103, 123, 114]
明文: flag{rsa_small_modulus}
用公钥再加密与原文一致: True
```

图 4.10: 第 4 章 C3 第 4 步: 逐字节解密得到 flag, 公钥再加密核对 True (真实终端实录)

结果与核对 明文是:

```
flag: flag{rsa_small_modulus}
```

核对有三层: $17 \times 2753 \bmod 3120 = 1$ 确认 d 是 e 的逆元; 往返测试 $65 \rightarrow 2790 \rightarrow 65$ 确认公式; 整段明文用公钥再加密得到的列表与 `rsa.txt` 逐项一致 (打印 True)。弱点说明: 这道题弱在模数太小, 试除就能分解出 p, q ; 真实 RSA 还会保证参数足够大并使用填充方案, 不能把本题的试除法当作普遍攻击。

为什么会成功 一句话机制: n 小到能分解, RSA 的安全根基就没了。展开说: RSA 的安全性建立在“分解 n 很难”之上; d 的计算需要 $\varphi(n) = (p-1)(q-1)$, 而分解出 p, q 就能算出 $\varphi(n)$ 。试除的复杂度是 $O(\sqrt{n})$, $n = 3233$ 时只有几十次; 换成 1024 位的 n , 试除次数是天文数字。另外注意“逐字节加密”这个出题设定: 它让每个密文整数只对应一个字节, 解密结果天然落在 0-255, 验证非常容易。

排错指南

- 解出的整数有大于 255 的: 说明“逐字节”假设或 d 算错了。先核对 $17d \bmod \varphi(n) = 1$, 再核对 $pq = n$;
- 解出来是乱码: 检查是否用了正确的密文列表 (别把 n, e 也读进去)、是否每项单独解密、是否整体转成大整数解密后再按字节序还原 (本题不是这种);
- `pow(17, -1, phi)` 报 `ValueError`: e 与 $\varphi(n)$ 不互素, 说明分解或 φ 算错了;
- 列表解析报错: `rsa.txt` 里 `ciphertext=` 后面才是列表, 用 `split('ciphertext=', 1)[1]` 切开再解析。

变式练习

1. 自己选两个小素数 (比如 $p = 101, q = 113$) 生成一组新的 (n, e) 和逐字节密文, 写成同款

rsa.txt，用本题流程解回来。

2. 把素数逐步增大 ($101 \rightarrow 1009 \rightarrow 10007$)，用 `time` 模块测每次试除分解耗时，观察成本随 $\sqrt{n}$ 上升的趋势。
3. 把逐字节加密改成“整段字节串按大端序转成一个大整数再模幂”，对比两种做法解密后的还原步骤有什么不同（提示：`int.to_bytes` 要指定长度和字节序）。

4.5 题 C4：同一模数的两份回执能否互相补全？（玄机 587）

题 C4 题面卡

题目来源：玄机平台 2026 安网杯-联号回执，<https://xj.edisec.net/challenges/587>；附件本地副本 labs/platform/attachments/587-linked-receipts.zip（离线可做到解出 flag）

给定材料：附件 ZIP 内 `export.json` 有两条记录：同一回执正文的两份 RSA 密文，`modulus` 相同（308 位十进制），`exponent` 分别是 65537 与 17；说明文件写明正文按 UTF-8 编码后转成大端无符号整数再模幂

目标：不分解大模数，利用两份密文之间的关系恢复正文

交付物：一个 `flag{...}` 字符串（32 位十六进制样式），在玄机平台对应题目页提交

考点：RSA 共模攻击：同 n 双互素指数 $\Rightarrow$ 扩展欧几里得组合出明文；负指数与模逆元

前置知识：2.5 节 RSA 三元组、2.7 节共模攻击；3.3 节 `pow`；3.4 节 `zipfile/json`

难度：★★★（挑战）

读题与思路 材料是两条回执记录：同一个模数 n 、指数分别是 65537 和 17、两份不同的密文。看到“同一明文、同一模数、两个指数”，应该立刻想到共模（2.7 节）；看到大模数（308 位十进制），应该立刻放弃分解。动手前先核对三个前提： $n_1 = n_2$ 吗？ $\gcd(e_1, e_2) = 1$ 吗？明文确实是同一份吗（说明文件写明“两条记录保护同一份回执正文”）？前提都成立，才能用 Bézout 系数组合。

分步操作与观察

1. 读附件，核对两条记录的公钥参数（图 4.11）：

```
python -c 'from zipfile import ZipFile; import json;
recs=json.loads(ZipFile(''labs/platform/attachments/587-linked-receipts.zip'')
               .read(''attachments/export.json''))['records'];
print(''记录数:'', len(recs));
[print(r[''receipt_id''], '', e='', r[''exponent''], '', n 的十进制位数='',
```

```
len(r['modulus']), ', 密文位数 =', len(r['ciphertext'])) for r in recs
];
print('两条记录模数相同:', recs[0]['modulus']==recs[1]['modulus']);
import math;
print('两个指数互素:', math.gcd(int(recs[0]['exponent']), int(recs[1]['exponent']))==1)
```

应该看到：两条记录 RS-2048-A ($e = 65537$) 与 RS-2048-B ($e = 17$)， n 都是 308 位十进制数，模数相同打印 True，指数互素打印 True。

2. 求 Bézout 系数：找 a, b 使 $65537a + 17b = 1$ （图 4.12）：

```
python -c 'import sys; sys.path.insert(0, 'labs/platform');
from solve_receipt import bezout;
g, a, b = bezout(65537, 17);
print('gcd(65537, 17) =', g, ', a =', a, ', b =', b);
print('验证 65537*a + 17*b =', 65537*a + 17*b)'
```

真实输出： $a = -8$ 、 $b = 30841$ ，代回验证得 1。负系数无妨： c_1^{-8} 表示先求 c_1 的模逆元再乘 8 次幂，Python 的 `pow(c1, -8, n)` 直接支持。

3. 运行复现脚本解出正文（图 4.13）：

```
python labs/platform/solve_receipt.py
python labs/platform/solve_receipt.py labs/platform/attachments/587-linked-
receipts.zip
```

应该看到三行输出：`coefficients: -8 30841`、完整正文、`flag: flag{f6d5...}`。脚本在输出前会做两件事：检查前提（模数相同、指数互素、密文与 n 互素）和再加密核对（把恢复的 m 分别用两个指数重新加密，必须等于附件里的两份密文），任一步不过都会抛异常而不是打印乱码。

4. 到玄机平台题目页 <https://xj.edisec.net/challenges/587> 提交 flag，确认页面显示“FLAG 正确”。

```
C4: 587 records share n, two exponents

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'from zipfile import
ZipFile; import json; recs=json.loads(ZipFile(''labs/platform/attachments/587-linked-receipts.zip'')
.read(''attachments/export.json''))[''records'']; print(''记录数:'', len(recs));
[print(r[''receipt_id''], '', e='', r[''exponent''], '', n 的十进制位数='', len(r[''modulus'']), '',
密文位数='', len(r[''ciphertext''])) for r in recs]; print(''两条记录模数相同:'',
recs[0][''modulus'']==recs[1][''modulus'']); import math; print(''两个指数互素:'',
math.gcd(int(recs[0][''exponent'']), int(recs[1][''exponent'']))==1)'
记录数: 2
RS-2048-A , e = 65537 , n 的十进制位数 = 308 , 密文位数 = 307
RS-2048-B , e = 17 , n 的十进制位数 = 308 , 密文位数 = 308
两条记录模数相同: True
两个指数互素: True
```

图 4.11: 第 4 章 C4 第 1 步: 587 两条记录同模数、指数 65537 与 17 互素 (真实终端实录)

```
C4: extended Euclid -> Bezout coefficients

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'import sys;
sys.path.insert(0, ''labs/platform''); from solve_receipt import bezout; g, a, b = bezout(65537,
17); print(''gcd(65537, 17) ='', g, '', a='', a, '', b='', b); print(''验证 65537*a + 17*b ='',
65537*a + 17*b)'
gcd(65537, 17) = 1 , a = -8 , b = 30841
验证 65537*a + 17*b = 1
PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python -c 'import math; c1, c2, n = 2,
3, 101; inv = pow(c1, -1, n); print(''负指数演示: c1^(-8) mod n 用 pow(c1, -8, n) ='', pow(c1, -8, n),
'', 与先求逆再乘幂一致:'', pow(inv, 8, n))'
负指数演示: c1^(-8) mod n 用 pow(c1, -8, n) = 58 , 与先求逆再乘幂一致: 58
```

图 4.12: 第 4 章 C4 第 2 步: 扩展欧几里得给出 $a=-8$ 、 $b=30841$, 负指数用模逆元处理 (真实终端实录)

```

C4: run solve_receipt.py -> flag

PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python labs/platform/solve_receipt.py
coefficients: -8 30841
message: 联考回执已核验。通行标记 flag{f6d5f18a35d8aa5802b12f7312b18b63}
flag: flag{f6d5f18a35d8aa5802b12f7312b18b63}
PS C:\Users\mzj\Desktop\CTF\CTF-training-problem-first-full> python labs/platform/solve_receipt.py
labs/platform/attachments/587-linked-receipts.zip
coefficients: -8 30841
message: 联考回执已核验。通行标记 flag{f6d5f18a35d8aa5802b12f7312b18b63}
flag: flag{f6d5f18a35d8aa5802b12f7312b18b63}

```

图 4.13: 第 4 章 C4 第 3 步: solve_receipt.py 输出正文与 flag (真实终端实录)

结果与核对 正文末尾的通行标记是:

flag: flag{f6d5f18a35d8aa5802b12f7312b18b63}

核对链: $\gcd(65537, 17) = 1$; $65537 \times (-8) + 17 \times 30841 = 1$; 恢复的 m 用两个指数再加密后与附件密文逐项一致 (脚本内建检查); 本账号此前在平台提交后显示正确反馈。

为什么会成功 一句话机制: 指数互素时, 两个模幂可以组合成一次“指数为 1”的模幂。展开说: $c_1 = m^{65537}$ 、 $c_2 = m^{17} \pmod{n}$, 而 $65537a + 17b = 1$, 所以 $c_1^a c_2^b = m^{65537a + 17b} = m$ 。攻击全程没有分解 n , 突破口是参数复用而不是大数分解。这与 C3 的“小模数可分解”是两种完全不同的弱点——看到 RSA 字样不能跳过附件条件检查直接套用某一种攻击。

排错指南

- 脚本报 the two records have different moduli: 两条记录不是共模, 共模推导不成立, 回题面核对附件版本;
- 脚本报 common-modulus conditions are not met: 指数不互素, 或某份密文与 n 不互素 ($\gcd(c, n) \neq 1$ 时该密文根本没被正确生成);
- 脚本报 re-encryption check failed: 算出的 m 不对, 检查 Bézout 系数方向 (a 配 e_1 、 b 配 e_2) 和负指数的处理;
- 解出乱码或缺字节: 整段字节串按大端还原时长度算错。用 `to_bytes((m.bit_length()+7)//8, 'big')` 这种按位长度换算, 不要随手丢零字节。

变式练习

1. 构造小规模共模练习: 取 $n = 3233$ 、明文一个字节, 分别用 $e_1 = 3$ 、 $e_2 = 5$ 加密, 用扩展

欧几里得找系数后解回明文，并做再加密核对。

2. 把两指数换成不互素的组合（如 $e_1 = 3$ 、 $e_2 = 9$ ）重做上题，观察推导在哪一步断掉，并解释为什么“不互素”意味着两个方程提供的信息不足以恢复 m 。
3. Bézout 系数不唯一：从 $a = -8, b = 30841$ 出发，构造另一组解（提示： $a' = a + 17t, b' = b - 65537t$ ），用它再解一次本题，验证结果相同。

第五章 复盘：知识地图与易错清单

本章目标：把五道题串成一张“下次遇到这类题怎么办”的地图，并把最容易踩的坑列成清单。

5.1 解题路线图：拿到材料先做什么

五道题的完整决策流程如下，建议自己默画一遍：

1. 第 1 步：识别表示。现在手里的是什么？

- 只有 0-9a-fA-F 且长度偶数 → 先 `bytes.fromhex` (C0);
- 文件里是十六进制文本 → 先 `Get-Content / xxd` 看真实内容 (C1);
- 一堆怪符号、种类有限 → 数种类，猜“每符号几位” (C2: 32 种 → 5 位);
- 若干个十进制大整数 + n, e → RSA 三元组 (C3、C4)。

2. 第 2 步：找可逆关系或弱点。

- 密钥空间小 → 穷举 (C1: 单字节 256 种);
- n 小 → 试除分解 (C3: $3233 = 61 \times 53$);
- 同 n 双 e 且互素 → 共模组合 (C4: $65537 \times (-8) + 17 \times 30841 = 1$);
- 自定义编码 → 用已知样本标定映射 (C2: 试报“夜班 OK”)。

3. 第 3 步：验证。

- 单字节异或 → 反向异或还原原密文;
- RSA → 用公钥再加密比对密文;
- 自定义编码 → 用已知明文验证映射;
- 平台题 → 提交后看“FLAG 正确”。

4. 第 4 步：写题解。保留输入版本、关键常数来源、算法、运行命令和平台反馈；把失败过的假设也写下来。

5.2 下次遇到这类题怎么办

题型信号	固定动作
一串十六进制字符	数字符数 $\rightarrow$ <code>bytes.fromhex</code> $\rightarrow$ 数字字节数 $\rightarrow$ 尝试 ASCII/UTF-8 解码。记录每一步长度。
一段不可读的字节、题面说“单字节密钥”	<code>bytes.fromhex</code> 后遍历 0-255，按“flag{ 前缀 + 全可打印”筛选；再用反向异或核对。
有限种符号组成的电文	数符号种类猜位数 ($2^5 = 32$)；找已知明文样本先验证映射；符号转比特、拼接、按 8 位分组。
(n, e, c) 且 n 很小	试除到 $\lfloor \sqrt{n} \rfloor$ 分解 $\rightarrow \varphi(n) \rightarrow \text{pow}(e, -1, \varphi(n)) \rightarrow \text{pow}(c, d, n)$ 。
同一明文、同一 n 、两个互素 e	扩展欧几里得找 a, b ，用 $c_1^a c_2^b \bmod n$ 组合；负指数走模逆元；最后再加密核对。
结果拿不准是不是 flag	按四条查：前缀、字符范围、结构（花括号配对）、独立再验证。

5.3 易错清单（按“错一下浪费半小时”的顺序）

1. 把十六进制文本当字节：`b'41'` 是 2 字节，`bytes.fromhex('41')` 才是 1 字节。动手前先数长度。
2. 读文件忘 `.strip()`：文件末尾的换行也是一个字节，会让异或和长度全错。
3. 对文本而不是字节做异或：先把十六进制解析成字节，再异或。
4. 只检查一个字符就下结论：短数据有很多“像真的”候选，要查完整前缀和整段字符范围。
5. 忘了 $\lfloor \sqrt{n} \rfloor$ 是试除上限：试到 56 为止 ($n = 3233$)，不是无限试。
6. $\varphi(n)$ 算错： $\varphi(n) = (p-1)(q-1)$ ，不是 $pq-1$ 。算完先验证 $ed \bmod \varphi(n) = 1$ 。
7. 看到 RSA 直接上某一种攻击：先检查附件条件—— n 小不小？ n 用没用过两次？指数互不互素？不同条件用不同攻击。
8. 共模攻击忘核对前提： $n_1 = n_2$ 、 $\gcd(e_1, e_2) = 1$ 、明文相同，缺一不可。
9. 负指数不会处理： $c^{-8} \bmod n$ 直接写 `pow(c, -8, n)` (Python 3.8+)，或先 `pow(c, -1, n)` 再乘幂。
10. 字节序还原错误：整体转整数的 RSA 要用 `int.to_bytes` 按大端还原，长度按位数换算，别乱丢零字节。

11. **验证环节省略**：脚本打印 flag 不等于对；再加密/反向运算/已知样本三类核对至少做一类，平台题还要看提交反馈。
12. **把猜测写成答案**：没被验证的假设写进题解时必须标注“未验证”，这是复盘能复用的前提。

一句话记忆
先识别表示，再找可逆关系，最后独立验证。三步都做完，答案才可信。

第六章 自测与参考答案

本章目标：先做自测题再看答案。答案里的推演刻意写全中间判断，直接抄最后一行是自欺。

6.1 课后自测题

1. `bytes.fromhex('666c6167')` 有几个字节？把它按 ASCII 读出来是什么？
2. 为什么说 Base64 是编码不是加密？举一个“编码没有密钥”的判断依据。
3. 写出异或的四行真值表，并用一行公式说明“加密 = 解密”。
4. 重复异或密钥长度为 3 时，密文的哪些位置共享同一个密钥字节？如何把这类题拆成子题？
5. 为什么单字节异或可以穷举？“可读性判断”至少要检查哪四条？
6. 用一句话定义模运算，并写出 Python 中计算 $b^e \bmod n$ 与 $a^{-1} \bmod n$ 的表达式。
7. RSA 的公钥三元组是什么？私钥指数 d 与 $\varphi(n)$ 是什么关系？
8. 什么条件下 e 在模 $\varphi(n)$ 下有逆元？如果不满足会怎样？
9. 为什么 $n = 3233$ 能被试除分解？试除的上限为什么是 $\lfloor \sqrt{n} \rfloor$ ？
10. 逐字节 RSA 与“整体转换成整数”的 RSA，在还原明文时有什么区别？
11. 共模攻击成立需要哪三个条件？写出组合明文的公式。
12. 五格电文为什么要先解公开试报？如果直接套标准 Base32 会发生什么？

6.2 参考答案与完整推演

第 1 题 4 个字节（8 个十六进制字符 $\div 2$ ），读出来是 flag。每个字节 2 个十六进制字符，所以字符数永远是字节数的两倍。

第 2 题 Base64 的转换规则完全公开，任何人不需要秘密信息就能解码；加密必须有密钥，没有密钥就还原不了。判断依据就是“是否需要秘密密钥”。同理十六进制、URL 编码都是编码。

第 3 题

x	k		$x \oplus k$
0	0		0
0	1		1
1	0		1
1	1		0

一行公式： $(x \oplus k) \oplus k = x$ 。异或自身可逆，所以加密程序即解密程序。

第 4 题 位置 0,3,6,9,... 共用 k_0 ；位置 1,4,7,... 共用 k_1 ；位置 2,5,8,... 共用 k_2 。按“下标 mod 3”把密文分成三组，每组就是一道单字节异或题，分别穷举后按原位置拼回，最后用反向异或核对整段。

第 5 题 因为密钥只有 1 个字节，取值 0-255 共 256 种，可以全部试完。四条检查：完整的 flag{ 前缀；字符是否全在可打印范围（32-126）；语言规律（英文单词/UTF-8 中文合法性）；结构合理性（花括号配对、长度合适）。

第 6 题 模运算 = 除以 n 取余数，结果在 0 到 $n-1$ 之间。Python: `pow(b, e, n)` 与 `pow(a, -1, n)`（要求 $\gcd(a, n) = 1$ ）。

第 7 题 三元组是 (n, e, c) ：模数、加密指数、密文。 d 是 e 模 $\varphi(n)$ 的逆元，即 $e \cdot d \equiv 1 \pmod{\varphi(n)}$ ； $\varphi(n) = (p-1)(q-1)$ ，其中 $n = pq$ 。

第 8 题 当且仅当 $\gcd(e, \varphi(n)) = 1$ 时 e 有模 $\varphi(n)$ 逆元。不满足则 d 不存在，标准 RSA 解密公式失效（出题人若故意这样设置，通常另有蹊跷，如 e 与 $\varphi(n)$ 共享因子时的特例分析，超出本册范围）。

第 9 题 $n = 3233$ 只有四位数， $\sqrt{3233} \approx 56.8$ ，最多试 55 个候选即可。上限是 $\lfloor \sqrt{n} \rfloor$ 因为 $n = p \times q$ 的两个因子不可能同时大于 $\sqrt{n}$ ：若 $p > \sqrt{n}$ 且 $q > \sqrt{n}$ ，则 $pq > n$ ，矛盾。

第 10 题 逐字节 RSA：每个密文整数独立对应一个明文字节，解密后直接 `bytes(...)` 组装，每个结果必须在 0-255。整体转整数的 RSA：解密得到一个大整数，要用 `int.to_bytes(length, 'big')` 按字节序还原整个字节串，长度要按位数换算、零字节不能乱丢。

第 11 题 三条件：同一明文；同一模数 n ； $\gcd(e_1, e_2) = 1$ 。公式：先求 a, b 使 $e_1a + e_2b = 1$ （扩展欧几里得），则 $m \equiv c_1^a \cdot c_2^b \pmod{n}$ ，负指数用模逆元。

第 12 题 公开试报是唯一能验证假设的材料：它同时校验符号表顺序、位序、分组方式和字符编码。直接套标准 Base32 会错在两处：字母表不对（这里的符号是自定义的 `glyph_plate`），填充规则未知。任何一步不匹配都应退回检查假设，而不是硬解正文。

6.3 完整推演：C3 的每一步为何必要

C3 附件给出 $n = 3233$ 、 $e = 17$ 和 23 个密文整数。先确认列表中每项都小于 n ，因为模幂的输出本应在 0 到 $n - 1$ 之间；这一步是合法性检查，防止把别的数据误当密文。再分解 n ：从 2 试除到 $\lfloor \sqrt{3233} \rfloor = 56$ ，找到 53，于是 $3233 = 61 \times 53$ 。因此 $\varphi(n) = 60 \times 52 = 3120$ 。求解 $17d \equiv 1 \pmod{3120}$ ，得 $d = 2753$ ；核对 $17 \times 2753 = 46801 = 15 \times 3120 + 1$ ，对 3120 取余确为 1。公式链闭合。

题目生成脚本提示“逐字节”，于是每个密文整数都对应一个明文字节。脚本中使用 `pow(c, d, n)` 计算模幂，比先计算巨大的 c^d 再取余高效且清晰。把所有结果放进 `bytes(...)` 后按 ASCII/UTF-8 解码，得到可读 flag。若某个结果超过 255，说明假设或参数有误；若得到乱码，先检查是否应整体解密、是否用错字节序、是否读错密文列表，而不是任意删字符。

```
from math import isqrt
n = 3233
factors = [(p, n // p) for p in range(2, isqrt(n) + 1)
            if n % p == 0]
print(factors)                # [(53, 61)]
phi = (61 - 1) * (53 - 1)     # 3120
d = pow(17, -1, phi)          # 2753
print(d, (17 * d) % phi)      # 2753 1
```

正常 RSA 不应选这样容易试除的模数。本题的目的在于把公钥、分解、模逆与解密串成可验证的链条，每一步都有独立核对点： $pq = n$ 、 $ed \bmod \varphi(n) = 1$ 、往返测试、再加密比对。

6.4 完整推演：C4 的推导为何成立

587 的两条记录共享模数 n ，指数 $e_1 = 65537$ 、 $e_2 = 17$ 。先核对 $\gcd(65537, 17) = 1$ ，于是扩展欧几里得算法给出 $65537 \times (-8) + 17 \times 30841 = 1$ 。由 $c_1 \equiv m^{65537}$ 、 $c_2 \equiv m^{17} \pmod{n}$ 可得

$$c_1^{-8} \cdot c_2^{30841} \equiv m^{65537 \times (-8) + 17 \times 30841} \equiv m \pmod{n}.$$

负指数表示模逆元： $c_1^{-8} = (c_1^{-1})^8$ ，计算前要确认 c_1 与 n 互素。得到 m 后按大端转换回字节并用 UTF-8 解码。最后重新计算两次加密，检查它们是否分别等于附件中的密文，避免“乱码里偶然出现 flag 样式文本”的误判。附带脚本会执行条件检查、解密和再加密核对：

```
python labs/platform/solve_receipt.py
```

答案：flag{f6d5f18a35d8aa5802b12f7312b18b63}，已在平台提交并获得正确反馈。这个案例与 C3 的“小模数可分解”突破口完全不同：只看到 RSA 字样，不能跳过附件条件检查直接套用一种攻击。

6.5 完整推演：C2 的验证链为何必要

586 的字模板有 32 个互不重复的符号，“每符号 5 位”成为可验证假设的依据是 $2^5 = 32$ 。去掉 sample_wire 中的分组空格，把每个符号替换为它在模板中的下标、用五位二进制写出，再从左到右每八位还原一个字节。这个方法解出 sample_plain 中的“夜班 OK”，说明符号顺序、位序与 UTF-8 解码均与已知样本相符；然后用完全相同的函数处理 dispatch_wire，得到完整正文，末尾包含所需 flag。

```
python labs/platform/solve_five_grid.py
```

答案：flag{c619020e518c599051efc804f5f84884}，已在平台提交并获得正确反馈。关键不是“看到 32 个字符就套 Base32”，而是用公开试报核对符号表、位序和字节编码；任何一步不匹配都应退回去检查假设。

附录 A：玄机平台真题状态与提交 checklist

6.6 本册涉及的平台题状态

题号与名称	本地材料	状态（照实记录，绝不编造）
586 五格电文	附件 <code>solve_five_grid.py</code>	ZIP、断网可解出并打印完整 flag；已在平台提交并获正确反馈。
587 联号回执	附件 <code>solve_receipt.py</code>	ZIP、断网可解出并打印完整 flag；已在平台提交并获正确反馈。

说明：平台题面、价格与界面可能变化，始终以当前题目详情页为准；本讲义不保存账号口令。提交后必须看到页面显示“FLAG 正确”及步骤完成，才能把答案标记为“已验证”。未验证的推测不得写成答案。

6.7 提交 checklist

1. 题目与链接是否记清：586 五格电文 <https://xj.edisec.net/challenges/586>，587 联号回执 <https://xj.edisec.net/challenges/587>；附件是否为当前官方版本，本地副本路径是否为 `labs/platform/attachments/586-five-grid.zip` 与 `labs/platform/attachments/587-linked-receipts.zip`。
2. 数学与结构前提是否核对：586 字模板 32 符号互不重复、试报解出“夜班 OK”；587 两条记录 modulus 相同、指数 65537 与 17 互素、Bézout 系数满足 $65537(-8) + 17(30841) = 1$ 。
3. 是否做再加密/反向核对：587 对恢复的 m 分别用两个指数重新加密，与附件两项密文逐项比较；586 用公开试报验证映射；C1 把明文重新异或；C3 把明文用公钥再加密。
4. 运行命令是否可移植：`pythonlabs/platform/solve_five_grid.py`、`pythonlabs/platform/solve_receipt.py`（也接受 ZIP 路径参数）；不在讲义与脚本中写账号口令，不打包复制平台附件给队友，只给链接与运行命令，由队友自行下载。

5. 平台反馈是否记录：提交后页面显示“FLAG 正确”及步骤完成；未验证的推测不写成答案，题解中保留已完成阶段与未完成步骤。

附录 B：命令速查表

两列表：左列“想做什么”，右列“敲什么命令”（Windows PowerShell；标注 WSL 的在 Ubuntu 里执行）。

想做什么	敲什么命令
查 Python 版本	<code>python --version</code>
跑一行 Python	<code>python -c 'print(2+3)'</code> （PowerShell 里单引号写两遍表示一个单引号）
跑脚本	<code>python labs/platform/solve_five_grid.py</code>
进交互模式	<code>python</code> （退出： <code>exit()</code> ）
看文件内容	<code>Get-Content labs/crypto/xor.hex</code> （WSL： <code>cat labs/crypto/xor.hex</code> ）
看文件字节（十六进制）	<code>xxd labs/crypto/xor.hex</code> （WSL）
十六进制转字节	<code>bytes.fromhex('515b5650')</code>
字节转十六进制	<code>b.hex()</code>
字符转字节	<code>'A'.encode('utf-8')</code>
字节转字符	<code>b'A'.decode('ascii')</code>
数字字符数/字节数	<code>len(s) / len(b)</code>
单字节异或	<code>bytes(x ^ k for x in data)</code>
算 $b^e \bmod n$	<code>pow(2, 10, 1000)</code>
求逆元 $a^{-1} \bmod n$	<code>pow(17, -1, 3120)</code>
判断互素	<code>math.gcd(65537, 17)</code>
试除分解小 n	<code>[p for p in range(2, math.isqrt(n)+1) if n%p==0]</code>
整数转字节串	<code>m.to_bytes((m.bit_length()+7)//8, 'big')</code>
字节串转整数	<code>int.from_bytes(b, 'big')</code>
列 ZIP 内容	<code>ZipFile(p).namelist()</code>
读 JSON 附件	<code>json.loads(z.read(p))</code>
解析 listing 形式数据	<code>ast.literal_eval(text)</code>

切换到资料根目录

```
cd C:/Users/你的用户名/Desktop/CTF 零基础自学资料包
```

WSL 里进资料目录

```
cd /mnt/c/Users/你的用户名/Desktop/CTF  
零基础自学资料包
```

附录 C：术语表

按拼音排序，每个术语一句话解释；页码见目录对应章节。

术语	一句话解释
ASCII	把 0–127 的数字分配给英文字母、数字、标点的老编码表，A 是 65。
Base64/Base32	把字节换成可打印字符的 编码 ，规则公开无密钥，不是加密。
Bézout 系数	使 $e_1a + e_2b = \gcd(e_1, e_2)$ 的整数对 (a, b) ，共模攻击的钥匙。
flag	CTF 题的答案，通常形如 <code>flag{...}</code> 。
哈希摘要	把任意数据映射成固定长度串的单向函数输出，如 32 位十六进制的 MD5。
可读性判断	用前缀、字符范围、语言、结构四条线索从候选中挑出真明文。
模运算 (mod)	除以 n 取余数，结果在 0 到 $n - 1$ 之间。
模逆元	满足 $a \cdot a^{-1} \equiv 1 \pmod{n}$ 的 a^{-1} ，要求 a 与 n 互素。
穷举 (brute force)	把所有可能的密钥全试一遍；只在密钥空间小时可行。
三元组 (n, e, c)	RSA 题最常见的公开材料：模数、加密指数、密文。
素数 (质数)	只能被 1 和自身整除的大于 1 的整数，RSA 用两个素数相乘造模数。
试除	从 2 逐个试除到 $\lfloor \sqrt{n} \rfloor$ 的分解方法，只对小 n 有效。
私钥指数 d	RSA 解密用的秘密指数， $e \cdot d \equiv 1 \pmod{\varphi(n)}$ 。
位序	多位二进制中高低位的排列顺序，自定义编码常在这里设坑。
小端序/大端序	多字节数据的存放顺序：低位在前叫小端，高位在前叫大端。
一次性密码本	与消息等长、真随机、只用一次的密钥，理论上不可破。
异或 (XOR)	按位“不同得 1”的运算， $(x \oplus k) \oplus k = x$ 。
字节	计算机存储的基本单位，一个 0–255 的数。
字节序	同“小端序/大端序”。
自定义编码	出题人自定符号表与位宽的表示方式，必须用已知样本验证映射。

写在最后 做 Crypto 题的三个动作请练到成为习惯：**识别表示、找可逆关系、独立验证**。下次拿到一串看不懂的数据，先别猜，先数长度、看字节、找结构。祝你解题顺利。